DBS Project Report

DATABASE SYSTEMS

Project Submission

Title: Gym Membership Management System

Technology Stack

Backend: FastAPI (Python) | Database: PostgreSQL

Frontend: React.js | API Documentation: Swagger UI

Group Members

Muhammad Abdullah 24k-0977

Saad Sohail 24k-0549

Zaid Ahmed 24k-0500

Abstract

The Gym Membership Management System is a comprehensive web-based application developed to streamline and automate the operations of a modern gym facility. The system provides complete functionality for member management, trainer scheduling, session booking, workout tracking, attendance recording, payment processing, and equipment maintenance. Built using FastAPI (Python) and PostgreSQL, the system features robust role-based access control for admins, trainers, and members.

The application ensures data integrity through proper database normalization (3NF), transactional operations, advanced SQL analytics, and set operations. The RESTful API backend exposes over 40 endpoints organized by functional module, while Pydantic models enforce strict data validation. Advanced analytics features include window functions, subqueries, UNION/INTERSECT/EXCEPT set operations, and running revenue totals.

I. Introduction

Background

The fitness industry requires sophisticated digital systems to manage memberships, track attendance, handle payments, and coordinate trainer-member interactions. Manual or partially digital systems often cause delays, scheduling conflicts, and inconsistent records. This project addresses these challenges by providing an integrated platform where members, trainers, and administrators can interact seamlessly.

Motivation

With growing gym memberships and the complexity of managing sessions, payments, and equipment simultaneously, there is a clear need for a centralized management solution. This project was motivated by the desire to apply database design principles in a real-world, practical scenario while building a fully functional API-driven web application.

Objectives

- Develop a centralized gym management database with proper normalization
- Implement secure, role-based access control for Admin, Trainer, and Member roles
- Automate member registration, session booking, and attendance tracking
- Provide real-time payment processing and overdue management
- Enable equipment maintenance tracking and alerts
- Deliver advanced analytics through SQL window functions, subqueries, and set operations
- Ensure data integrity through transaction management and constraint enforcement

Scope

The system covers the following modules:

- User authentication and role-based access
- Member and trainer management
- Membership plan configuration
- Session scheduling and booking
- Workout logging and history
- Attendance check-in tracking
- Payment recording and revenue analytics
- Equipment inventory and maintenance
- Admin dashboard with aggregated statistics

Limitations

- No external payment gateway integration (payments recorded manually)
- Password hashing uses plain-text storage in the prototype (bcrypt recommended for production)
- No real-time notifications or push alerts
- Single-location system without multi-branch support

II. System Requirements & Analysis

Functional Requirements

- User registration and authentication with role assignment
- Role-based access: Admin, Trainer, and Member
- Full CRUD operations on members, trainers, plans, sessions, and equipment
- Session booking, cancellation, and completion tracking
- Attendance check-in with timestamp logging
- Payment recording with status management (paid, pending, overdue)
- Overdue payment detection and automated account freezing via transactions
- Revenue reporting by method, plan, and time period
- Trainer workload analytics and session fill rate reporting
- Equipment maintenance alerts based on condition and date

Non-Functional Requirements

- RESTful API with Swagger documentation
- Pydantic models for input validation and response serialization
- CORS middleware for cross-origin React frontend support
- Error handling with HTTP exception codes and rollback on failure
- Parameterized queries to prevent SQL injection
- PostgreSQL for ACID-compliant transaction support

User Roles

Admin

- Full access to all modules and endpoints
- Manage members, trainers, sessions, and equipment
- View and process payments and revenue analytics
- Access admin dashboard with aggregated statistics
- Register members and trainers via atomic transactions
- Process overdue payments and freeze inactive accounts

Trainer

- View personal session schedule
- Log workouts for assigned members
- View assigned member workout history

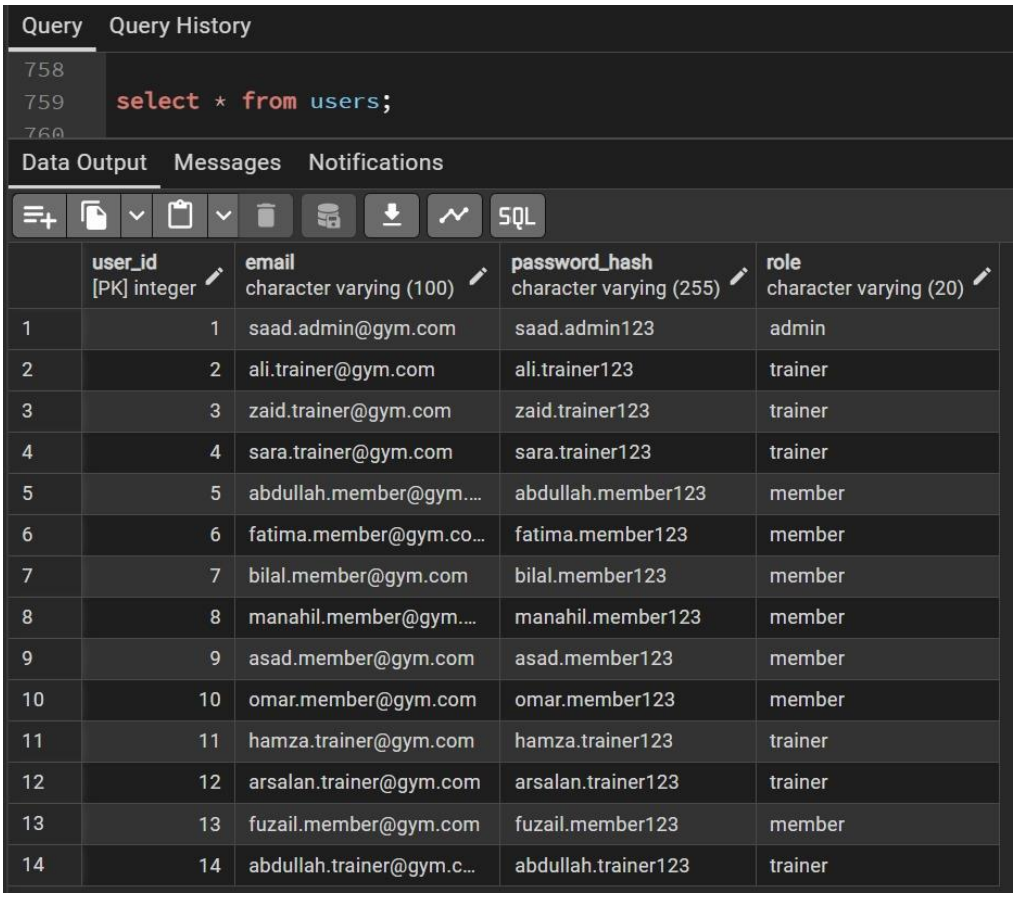
Member

- Browse upcoming sessions and book/cancel sessions
- View personal workout and attendance history
- Check membership plan and payment history
- Access personal dashboard with stats

III. Data Dictionary

USERS Table

Field	Type	Constraints	Description
user_id (PK)	SERIAL	PRIMARY KEY	Auto-incremented unique user identifier
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email address
password_hash	VARCHAR(255)	NOT NULL	Password (hashed in production)
role	VARCHAR(20)	CHECK (admin/trainer/member)	User role for access control



Query Query History

```
758
759  select * from users;
760
```

Data Output Messages Notifications

	user_id [PK] integer	email character varying (100)	password_hash character varying (255)	role character varying (20)
1	1	saad.admin@gym.com	saad.admin123	admin
2	2	ali.trainer@gym.com	ali.trainer123	trainer
3	3	zaid.trainer@gym.com	zaid.trainer123	trainer
4	4	sara.trainer@gym.com	sara.trainer123	trainer
5	5	abdullah.member@gym....	abdullah.member123	member
6	6	fatima.member@gym.co...	fatima.member123	member
7	7	bilal.member@gym.com	bilal.member123	member
8	8	manahil.member@gym....	manahil.member123	member
9	9	asad.member@gym.com	asad.member123	member
10	10	omar.member@gym.com	omar.member123	member
11	11	hamza.trainer@gym.com	hamza.trainer123	trainer
12	12	arsalan.trainer@gym.com	arsalan.trainer123	trainer
13	13	fuzail.member@gym.com	fuzail.member123	member
14	14	abdullah.trainer@gym.c...	abdullah.trainer123	trainer

MEMBERS Table

Field	Type	Constraints	Description
member_id (PK)	SERIAL	PRIMARY KEY	Unique member identifier
user_id (FK)	INT	UNIQUE, REFERENCES USERS	Links to login account
first_name	VARCHAR(50)	NOT NULL	Member's first name
last_name	VARCHAR(50)	NOT NULL	Member's last name
phone	VARCHAR(15)	OPTIONAL	Contact number
join_date	DATE	DEFAULT CURRENT_DATE	Membership start date
plan_id (FK)	INT	REFERENCES MEMBERSHIP_PLANS	Current active plan
status	VARCHAR(20)	CHECK (active/inactive/frozen)	Account status

```
Query  Query History
758
759  select * from members;
760
```

Data Output | Messages | Notifications

Showing rows: 1 to 7

	member_id [PK] integer	user_id integer	first_name character varying (50)	last_name character varying (50)	phone character varying (15)	join_date date	plan_id integer	status character varying (20)
1	1	5	Abdullah	Choudhry	03456235788	2024-10-01	2	active
2	2	6	Fatima	Amjad	03466255856	2024-11-15	2	active
3	4	8	Manahil	Mahmood	03476878787	2024-08-20	2	active
4	6	10	Omar	Sheikh	03001112233	2026-05-10	1	active
5	5	9	Asad	Ahmed	03214589843	2025-10-06	2	inactive
6	7	13	Fuzail	Raza	03178820402	2026-05-12	3	active
7	3	7	Bilal	Khan	03002425678	2025-01-10	4	active

TRAINERS Table

Field	Type	Constraints	Description
trainer_id (PK)	SERIAL	PRIMARY KEY	Unique trainer identifier
user_id (FK)	INT	UNIQUE, REFERENCES USERS	Links to login account
first_name	VARCHAR(50)	NOT NULL	Trainer's first name
last_name	VARCHAR(50)	NOT NULL	Trainer's last name
specialization	VARCHAR(100)	OPTIONAL	Training expertise area
hire_date	DATE	DEFAULT CURRENT_DATE	Date joined the gym
phone	VARCHAR(15)	OPTIONAL	Contact number

```
Query  Query History
758
759  select * from trainers;
760
```

Data Output | Messages | Notifications

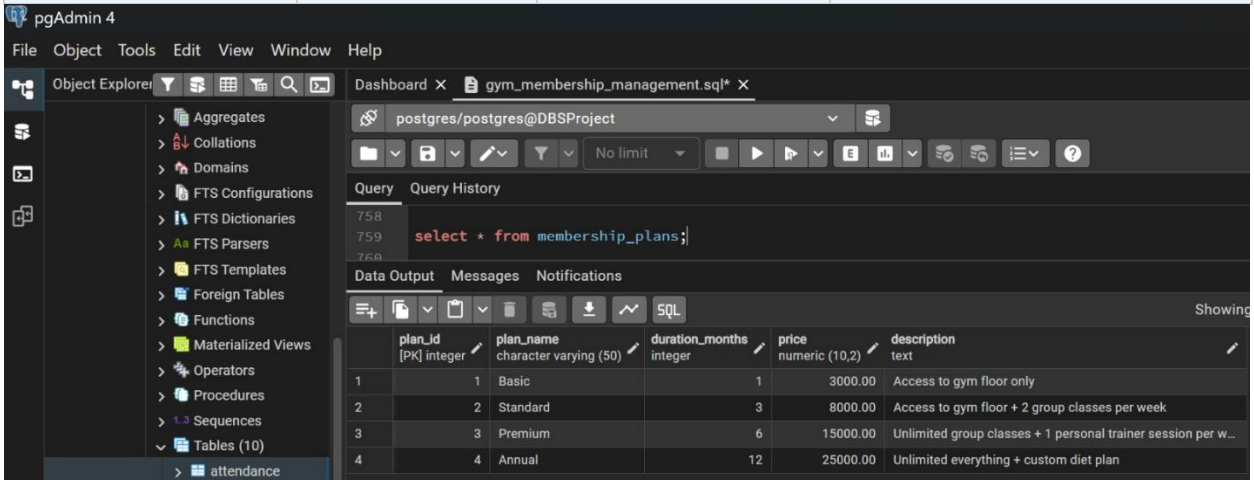
Showing ro

	trainer_id [PK] integer	user_id integer	first_name character varying (50)	last_name character varying (50)	specialization character varying (100)	hire_date date	phone character varying (15)
1	2	3	Zaid	Ahmed	Cardio & HIIT	2023-08-15	03459876543
2	3	4	Sara	Khursheed	Yoga & Flexibility	2024-01-10	03459876543
3	1	2	Ali	Naqvi	Strength & Conditioning	2023-05-01	03459876543
4	4	11	Hamza	Ali	CrossFit	2026-05-10	03119988776
5	5	12	Arsalan	Rizvi	CrossFit	2026-05-10	03119988776
6	6	14	Abdullah	Omer	Powerlifting	2026-05-12	03467893357

MEMBERSHIP_PLANS Table

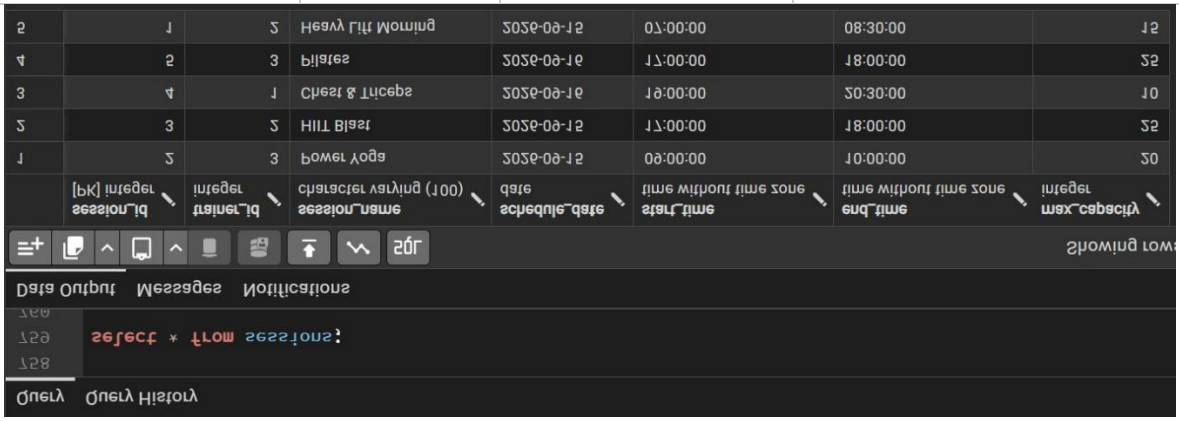
Field	Type	Constraints	Description
plan_id (PK)	SERIAL	PRIMARY KEY	Unique plan identifier

Field	Type	Constraints	Description
plan_name	VARCHAR(50)	NOT NULL	Plan title (e.g., Basic, Premium)
duration_months	INT	NOT NULL	Plan duration in months
price	DECIMAL(10,2)	NOT NULL	Plan price in PKR
description	TEXT	OPTIONAL	Features included in the plan



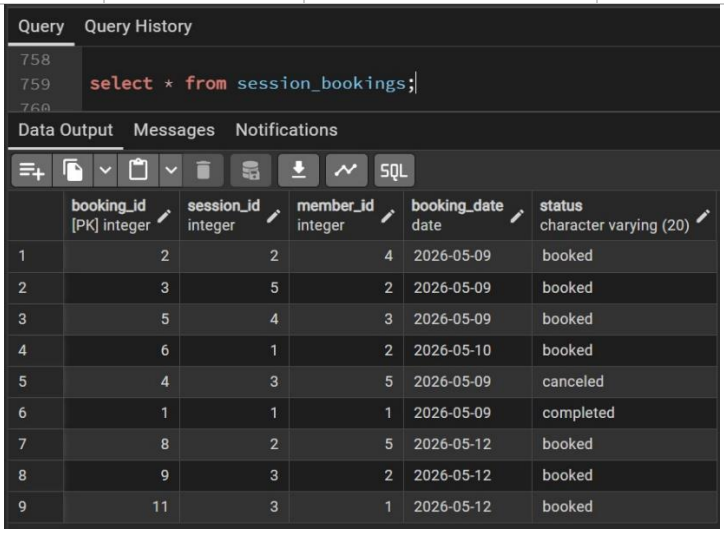
SESSIONS Table

Field	Type	Constraints	Description
session_id (PK)	SERIAL	PRIMARY KEY	Unique session identifier
trainer_id (FK)	INT	REFERENCES TRAINERS	Assigned trainer
session_name	VARCHAR(100)	NOT NULL	Name/type of session
schedule_date	DATE	NOT NULL	Date of the session
start_time / end_time	TIME	NOT NULL	Session time window
max_capacity	INT	DEFAULT 20	Maximum participants allowed



SESSION_BOOKINGS Table

Field	Type	Constraints	Description
booking_id (PK)	SERIAL	PRIMARY KEY	Unique booking record
session_id (FK)	INT	REFERENCES SESSIONS	Booked session
member_id (FK)	INT	REFERENCES MEMBERS	Booking member
booking_date	DATE	DEFAULT CURRENT_DATE	Date booking was made
status	VARCHAR(20)	CHECK (booked/canceled/completed)	Current booking state
(session_id, member_id)	UNIQUE	Composite Constraint	Prevents double booking

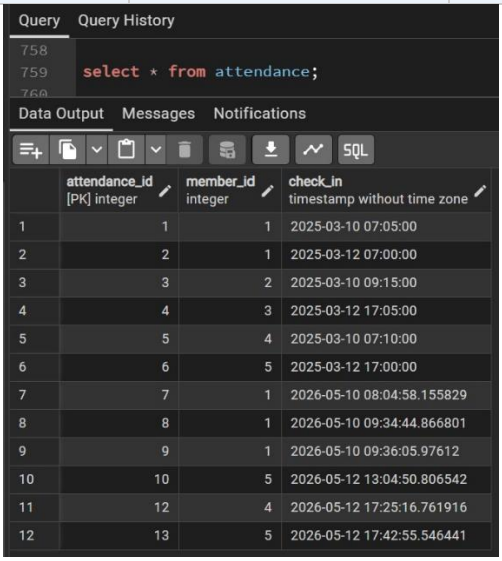


PAYMENTS Table

Field	Type	Constraints	Description
payment_id (PK)	SERIAL	PRIMARY KEY	Unique payment record
member_id (FK)	INT	REFERENCES MEMBERS	Paying member
amount	DECIMAL(10,2)	NOT NULL	Payment amount in PKR
payment_date	DATE	DEFAULT CURRENT_DATE	Date of payment
payment_method	VARCHAR(50)	DEFAULT 'cash'	Cash or Bank Transfer
status	VARCHAR(20)	CHECK (paid/pending/overdue)	Payment status

ATTENDANCE Table

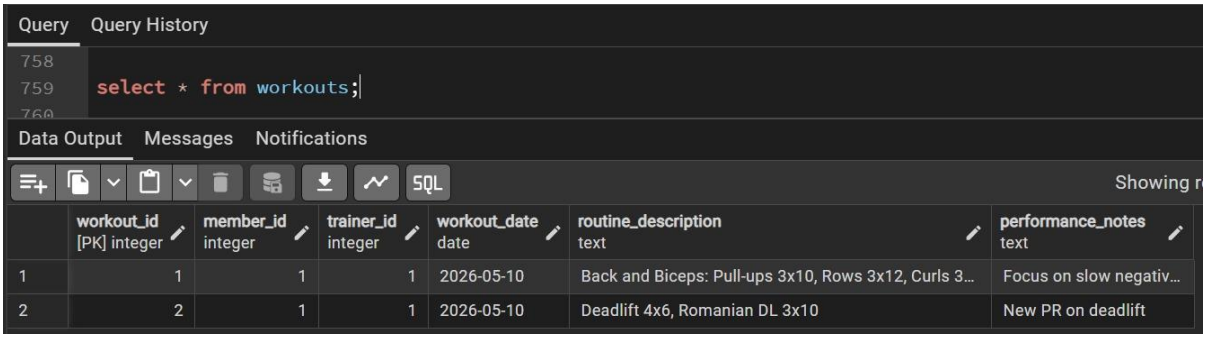
Field	Type	Constraints	Description
attendance_id (PK)	SERIAL	PRIMARY KEY	Unique check-in record
member_id (FK)	INT	REFERENCES MEMBERS	Member who checked in
check_in	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Exact check-in date and time



	attendance_id [PK] integer	member_id integer	check_in timestamp without time zone
1	1	1	2025-03-10 07:05:00
2	2	1	2025-03-12 07:00:00
3	3	2	2025-03-10 09:15:00
4	4	3	2025-03-12 17:05:00
5	5	4	2025-03-10 07:10:00
6	6	5	2025-03-12 17:00:00
7	7	1	2026-05-10 08:04:58.155829
8	8	1	2026-05-10 09:34:44.866801
9	9	1	2026-05-10 09:36:05.97612
10	10	5	2026-05-12 13:04:50.806542
11	12	4	2026-05-12 17:25:16.761916
12	13	5	2026-05-12 17:42:55.546441

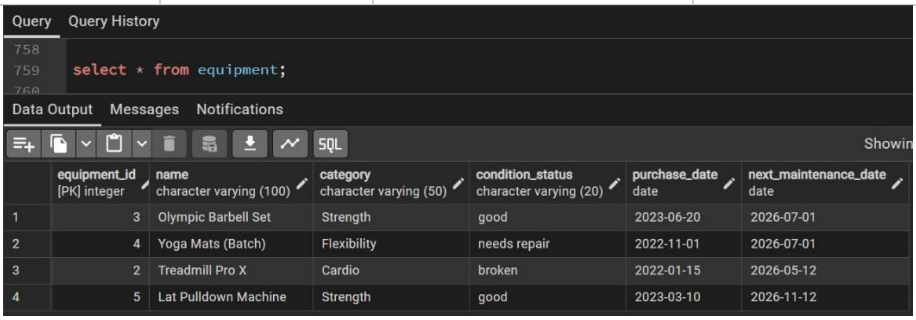
WORKOUTS Table

Field	Type	Constraints	Description
workout_id (PK)	SERIAL	PRIMARY KEY	Unique workout log
member_id (FK)	INT	REFERENCES MEMBERS	Member who performed workout
trainer_id (FK)	INT	REFERENCES TRAINERS	Trainer who assigned it
workout_date	DATE	DEFAULT CURRENT_DATE	Date workout was performed
routine_description	TEXT	NOT NULL	Exercises and sets performed
performance_notes	TEXT	OPTIONAL	Notes on performance and PRs



EQUIPMENT Table

Field	Type	Constraints	Description
equipment_id (PK)	SERIAL	PRIMARY KEY	Unique equipment record
name	VARCHAR(100)	NOT NULL	Equipment name
category	VARCHAR(50)	OPTIONAL	Type (Cardio, Strength, etc.)
condition_status	VARCHAR(20)	CHECK (good/needs repair/broken)	Current condition
purchase_date	DATE	OPTIONAL	Date of purchase
next_maintenance_date	DATE	OPTIONAL	Upcoming maintenance deadline



IV. Database Design & Normalization

Normalization (3NF)

All tables in the database are normalized to Third Normal Form (3NF):

- 1NF: All attributes contain atomic values with no repeating groups. Each table has a well-defined primary key.
- 2NF: All non-key attributes are fully functionally dependent on the entire primary key. Composite keys (e.g., session_id + member_id in SESSION_BOOKINGS) are only used where necessary.

- 3NF: No transitive dependencies exist. For example, plan details are stored in MEMBERSHIP_PLANS and referenced via plan_id in MEMBERS, rather than duplicating plan data in each member record.

Relationships

Relationship	Type	Constraint
USERS → MEMBERS	One-to-One	ON DELETE CASCADE
USERS → TRAINERS	One-to-One	ON DELETE CASCADE
MEMBERSHIP_PLANS → MEMBERS	One-to-Many	ON DELETE SET NULL
TRAINERS → SESSIONS	One-to-Many	ON DELETE CASCADE
SESSIONS → SESSION_BOOKINGS	One-to-Many	ON DELETE CASCADE
MEMBERS → SESSION_BOOKINGS	One-to-Many	ON DELETE CASCADE
MEMBERS → ATTENDANCE	One-to-Many	ON DELETE CASCADE
MEMBERS → WORKOUTS	One-to-Many	ON DELETE CASCADE
TRAINERS → WORKOUTS	One-to-Many	ON DELETE SET NULL
MEMBERS → PAYMENTS	One-to-Many	ON DELETE CASCADE

V. Database Implementation

Sample Data

The database is seeded with 1 admin, 3 trainers, and 5 members to demonstrate functionality:

Membership Plans

Plan	Duration	Price (PKR)	Description
Basic	1 month	3,000	Access to gym floor only
Standard	3 months	8,000	Gym floor + 2 group classes/week
Premium	6 months	15,000	Unlimited classes + 1 PT session/week
Annual	12 months	25,000	Unlimited everything + custom diet plan

Trainers

Name	Specialization	Hire Date
Ali Naqvi	Strength & Conditioning	2023-05-01
Zaid Ahmed	Cardio & HIIT	2023-08-15
Sara Khursheed	Yoga & Flexibility	2024-01-10

Upcoming Sessions

Session Name	Trainer	Date	Time	Capacity
Heavy Lift Morning	Ali Naqvi	2026-09-15	07:00 - 08:30	15
Power Yoga	Sara Khursheed	2026-09-15	09:00 - 10:00	20
HIIT Blast	Zaid Ahmed	2026-09-15	17:00 - 18:00	25
Chest & Triceps	Ali Naqvi	2026-09-16	19:00 - 20:30	10
Pilates	Sara Khursheed	2026-09-16	17:00 - 18:00	25

VI. Key SQL Queries

Authentication

Login query that uses LEFT JOINs to retrieve role-specific IDs for members and trainers:

```
SELECT u.user_id, u.role, u.password_hash,
       m.member_id, m.first_name AS member_first_name,
       t.trainer_id, t.first_name AS trainer_first_name
FROM users u
LEFT JOIN members m ON u.user_id = m.user_id
LEFT JOIN trainers t ON u.user_id = t.user_id
WHERE u.email = %s;
```

Member Dashboard (Scalar Subqueries)

Aggregates a member's plan details, total paid, and total visit count in a single query:

```
SELECT m.first_name || ' ' || m.last_name AS full_name,
       mp.plan_name, mp.duration_months,
       (SELECT SUM(amount) FROM payments
        WHERE member_id = m.member_id AND status = 'paid') AS total_paid,
       (SELECT COUNT(*) FROM attendance
        WHERE member_id = m.member_id) AS total_visits
FROM members m
JOIN membership_plans mp ON m.plan_id = mp.plan_id
WHERE m.member_id = %s;
```

Sessions with Remaining Spots

Calculates available spots dynamically by subtracting booked count from capacity:

```
SELECT s.session_id, s.session_name,
       t.first_name || ' ' || t.last_name AS trainer,
       s.max_capacity,
       s.max_capacity - COUNT(sb.booking_id) AS spots_remaining
FROM sessions s
JOIN trainers t ON s.trainer_id = t.trainer_id
LEFT JOIN session_bookings sb
       ON s.session_id = sb.session_id AND sb.status = 'booked'
GROUP BY s.session_id, t.first_name, t.last_name, s.max_capacity
ORDER BY s.schedule_date, s.start_time;
```

Admin Dashboard Stats (Multi-Subquery)

Returns all dashboard statistics in a single query using correlated subqueries:

```
SELECT
  (SELECT COUNT(*) FROM members WHERE status = 'active')        AS
active_members,
  (SELECT COUNT(*) FROM trainers)                                AS
total_trainers,
  (SELECT COUNT(*) FROM sessions WHERE schedule_date >= CURRENT_DATE) AS
upcoming_sessions,
  (SELECT COALESCE(SUM(amount), 0) FROM payments
   WHERE status = 'paid')                                        AS
total_revenue,
  (SELECT COUNT(*) FROM payments
   WHERE status IN ('pending','overdue'))                        AS
pending_payments,
  (SELECT COUNT(*) FROM equipment
   WHERE condition_status != 'good')                             AS
equipment_issues;
```

Revenue per Payment Method

```
SELECT payment_method,
       COUNT(*) AS total_transactions,
       SUM(amount) AS total_collected
FROM payments
WHERE status = 'paid'
GROUP BY payment_method;
```

Monthly Revenue (Last 6 Months)

```
SELECT TO_CHAR(payment_date, 'YYYY-MM') AS month,
       SUM(amount) AS total_revenue
```

```
FROM payments
WHERE status = 'paid'
  AND payment_date >= CURRENT_DATE - INTERVAL '6 months'
GROUP BY TO_CHAR(payment_date, 'YYYY-MM')
ORDER BY month ASC;
```

Session Fill Rate Analytics

```
SELECT s.session_name, s.schedule_date,
       s.max_capacity,
       COUNT(sb.booking_id) AS booked,
       ROUND(COUNT(sb.booking_id) * 100.0 / s.max_capacity, 1) AS fill_pct
FROM sessions s
LEFT JOIN session_bookings sb
       ON s.session_id = sb.session_id AND sb.status = 'booked'
GROUP BY s.session_id, s.session_name, s.schedule_date, s.max_capacity
ORDER BY fill_pct DESC;
```

Members Who Never Checked In (Subquery)

```
SELECT m.first_name || ' ' || m.last_name AS member, u.email
FROM members m
JOIN users u ON m.user_id = u.user_id
WHERE m.member_id NOT IN (
    SELECT DISTINCT member_id FROM attendance
);
```

Payments Above Average (Subquery)

```
SELECT m.first_name || ' ' || m.last_name AS member, p.amount
FROM payments p
JOIN members m ON p.member_id = m.member_id
WHERE p.amount > (SELECT AVG(amount) FROM payments WHERE status = 'paid')
ORDER BY p.amount DESC;
```

VII. Advanced SQL: Set Operations & Window Functions

Set Operations

UNION: All Interacting Members

Returns every member who has either attended the gym or booked a session, eliminating duplicate member IDs:

```
SELECT member_id, 'attendance' AS activity FROM attendance
UNION
```

```
SELECT member_id, 'booking' AS activity FROM session_bookings;
```

INTERSECT: Highly Engaged Members

Finds members who both attended the gym AND have a workout logged - indicating high engagement:

```
SELECT member_id FROM attendance
INTERSECT
SELECT member_id FROM workouts;
```

EXCEPT: Active Members Who Never Visited

Identifies active paying members who have never checked in (PostgreSQL uses EXCEPT instead of MINUS):

```
SELECT member_id FROM members WHERE status = 'active'
EXCEPT
SELECT DISTINCT member_id FROM attendance;
```

EXCEPT: Incomplete Bookings

Members who booked sessions but never completed one:

```
SELECT member_id FROM session_bookings WHERE status = 'booked'
EXCEPT
SELECT member_id FROM session_bookings WHERE status = 'completed';
```

Window Functions

RANK(): Attendance Leaderboard

Ranks all members by total gym visits. RANK() handles ties - two members with 10 visits both receive Rank 1, and the next receives Rank 3:

```
SELECT m.first_name || ' ' || m.last_name AS member,
       COUNT(a.attendance_id) AS visits,
       RANK() OVER (ORDER BY COUNT(a.attendance_id) DESC) AS rank
FROM attendance a
JOIN members m ON a.member_id = m.member_id
GROUP BY m.member_id, m.first_name, m.last_name;
```

SUM() OVER: Running Revenue Total

Calculates a cumulative running total of revenue over time, allowing tracking of revenue growth day by day:

```
SELECT payment_date, amount,
       SUM(amount) OVER (ORDER BY payment_date) AS running_total
```

```
FROM payments
WHERE status = 'paid'
ORDER BY payment_date;
```

Plan Pricing Breakdown (Arithmetic)

Calculates monthly rate and 10% discounted price dynamically for each plan:

```
SELECT plan_name,
       price AS total_price,
       ROUND(price / duration_months, 2) AS monthly_rate,
       ROUND(price * 0.10, 2) AS ten_pct_discount,
       ROUND(price - (price * 0.10), 2) AS discounted_price
FROM membership_plans;
```

VIII. Transaction Management

All critical multi-step operations are wrapped in database transactions to ensure atomicity. If any step fails, the entire operation is rolled back.

Register New Member (Atomic User + Member Creation)

Creates a user account and member profile in a single transaction. Uses RETURNING to capture the auto-generated user_id for the second insert:

```
BEGIN;
INSERT INTO users (email, password_hash, role)
VALUES ('omar.member@gym.com', 'hashed_pass', 'member')
RETURNING user_id;  -- captures new ID

INSERT INTO members (user_id, first_name, last_name, phone, plan_id)
VALUES (<returned_id>, 'Omar', 'Sheikh', '03001112233', 1);
COMMIT;
```

Session Check-In (Booking + Attendance)

Simultaneously marks a booking as completed AND logs gym attendance:

```
BEGIN;
UPDATE session_bookings SET status = 'completed'
WHERE member_id = %s AND session_id = %s;

INSERT INTO attendance (member_id) VALUES (%s);
COMMIT;
```

Process Overdue Payments (Bulk Update + Freeze)

Marks pending payments older than 30 days as overdue, then freezes those members' accounts:

```
BEGIN;
UPDATE payments SET status = 'overdue'
WHERE status = 'pending'
  AND payment_date < CURRENT_DATE - INTERVAL '30 days';

UPDATE members SET status = 'frozen'
WHERE member_id IN (
    SELECT DISTINCT member_id FROM payments WHERE status = 'overdue'
) AND status = 'active';
COMMIT;
```

Assign New Plan (Plan Update + Payment Record)

Updates a member's active plan and simultaneously records the payment for it:

```
BEGIN;
UPDATE members SET plan_id = %s WHERE member_id = %s;

INSERT INTO payments (member_id, amount, payment_date, payment_method, status)
VALUES (%s, %s, CURRENT_DATE, %s, 'paid');
COMMIT;
```

Delete Member (Cascade + Booking Cancellation)

Cancels all active bookings before deleting the user account. The ON DELETE CASCADE constraint then removes the member record automatically:

```
BEGIN;
UPDATE session_bookings SET status = 'canceled'
WHERE member_id = %s;

DELETE FROM users
WHERE user_id = (SELECT user_id FROM members WHERE member_id = %s);
-- CASCADE removes MEMBERS record automatically
COMMIT;
```

Triggers:

If a member is frozen/inactive and a 'paid' payment is recorded, they automatically become 'active'

```
CREATE OR REPLACE FUNCTION fn_auto_activate_member()
RETURNS TRIGGER AS $$ BEGIN
   IF NEW.status = 'paid' THEN
      UPDATE members SET status = 'active' WHERE member_id = NEW.member_id AND
status != 'active';
```

```
    END IF;
    RETURN NEW;
END;


DROP TRIGGER IF EXISTS trg_auto_activate_member ON payments;
CREATE TRIGGER trg_auto_activate_member
AFTER INSERT ON payments
FOR EACH ROW
EXECUTE FUNCTION fn_auto_activate_member();


-- When a booking status changes to 'completed', automatically log a check-in
CREATE OR REPLACE FUNCTION fn_auto_attendance()
RETURNS TRIGGER AS $$ BEGIN
    IF NEW.status = 'completed' AND (OLD.status IS NULL OR OLD.status != 'completed')
THEN
        INSERT INTO attendance (member_id, check_in)
        VALUES (NEW.member_id, CURRENT_TIMESTAMP);
    END IF;
    RETURN NEW;
END;
 ;

DROP TRIGGER IF EXISTS trg_auto_attendance ON session_bookings;
CREATE TRIGGER trg_auto_attendance
AFTER UPDATE ON session_bookings
FOR EACH ROW
EXECUTE FUNCTION fn_auto_attendance();
```

Views:

Combines members, plans, payments, and attendance into one easy-to-read table

```
CREATE OR REPLACE VIEW vw_member_dashboard AS
SELECT
    m.member_id,
    m.first_name || ' ' || m.last_name AS full_name,
    mp.plan_name,
    m.status,
    COALESCE((SELECT SUM(p.amount) FROM payments p WHERE p.member_id =
m.member_id AND p.status = 'paid'), 0) AS total_paid,
    (SELECT COUNT(a.attendance_id) FROM attendance a WHERE a.member_id =
m.member_id) AS total_visits
FROM members m
LEFT JOIN membership_plans mp ON m.plan_id = mp.plan_id;

-- Calculates spots remaining dynamically
CREATE OR REPLACE VIEW vw_session_capacity AS
```

```
SELECT
    s.session_id,
    s.session_name,
    t.first_name || ' ' || t.last_name AS trainer,
    s.schedule_date,
    s.max_capacity,
    COUNT(sb.booking_id) AS current_bookings,
    s.max_capacity - COUNT(sb.booking_id) AS spots_remaining
FROM sessions s
JOIN trainers t ON s.trainer_id = t.trainer_id
LEFT JOIN session_bookings sb ON s.session_id = sb.session_id AND sb.status =
'booked'
GROUP BY s.session_id, t.first_name, t.last_name;
```

Summary of Transactions

Transaction	Operations	Rollback On
Register Member	INSERT users + INSERT members	Email duplicate, invalid plan_id
Register Trainer	INSERT users + INSERT trainers	Email duplicate, constraint violation
Session Check-In	UPDATE booking status + INSERT attendance	Invalid session/member ID
Process Overdue	UPDATE payments + UPDATE members	Database lock or constraint error
Assign Plan	UPDATE member plan + INSERT payment	Invalid plan_id, payment error
Deactivate Member	UPDATE member status + UPDATE bookings	Member not found
Delete Member	UPDATE bookings + DELETE user	Foreign key violation

IX. Application & API

Technology Stack

Component	Technology	Purpose
Backend Framework	FastAPI (Python)	REST API with auto-generated Swagger docs

Component	Technology	Purpose
Database	PostgreSQL	Relational data storage with ACID transactions
Data Validation	Pydantic v2	Request/response model validation
Frontend	React.js	User interface for all roles
API Documentation	Swagger UI	Interactive endpoint testing
Connection	psycopg2	Python-PostgreSQL database driver
CORS	FastAPI Middleware	Allows React dev server cross-origin requests

API Endpoint Summary

Module	Endpoints	Key Operations
Authentication	1	Login with role-based response
Members	3	List, status update, dashboard
Trainers	4	List, schedule, session count, workload
Plans	2	List, create custom plan
Sessions	8	Upcoming, spots, fill rate, booking, cancellation
Workouts	4	Log, list all, member history, trainer history
Attendance	6	Check-in, log, active members, lazy members
Payments	10	Record, list, overdue, revenue analytics
Equipment	4	List, maintenance alerts, condition update, repair
Dashboard	1	Aggregated admin statistics
Transactions	8	Atomic multi-step operations
Advanced Analytics	8	Set ops, window functions, subqueries

Role-Based Access Design

The system enforces three distinct roles at the API level. Admin credentials are validated through the login endpoint, and the returned role field is used by the React frontend to conditionally render admin-only sections. Trainer and member endpoints are filtered by their respective trainer_id or member_id parameters from the login response.

X. Testing & Results

Functional Testing

Module	Test Case	Result
Authentication	Login with correct admin credentials	PASS
Authentication	Login with wrong password	401 Unauthorized returned
Members	Register member with existing email	400 conflict raised
Sessions	Book same session twice for same member	400 duplicate booking caught
Sessions	View sessions with remaining spots	PASS - accurate count
Attendance	Check in a member twice on same day	Both timestamps recorded
Payments	Process overdue payments	Pending payments updated, accounts frozen
Transactions	Delete member with active bookings	Bookings cancelled, user deleted
Equipment	Mark equipment as repaired	Status updated, maintenance rescheduled
Analytics	Revenue running total	PASS - cumulative sum verified

Observations

- Role-based access correctly restricts trainer and member data to their own records
- The UNIQUE constraint on (session_id, member_id) successfully prevents double bookings
- ON DELETE CASCADE reliably removes dependent records when a user account is deleted
- Transactions ensure that partial failures (e.g., second INSERT failing) roll back cleanly
- Window functions produce accurate rankings and running totals across all payment records
- EXCEPT queries correctly identify members with no attendance history

XI. Results & Discussion

Achievements

- Successfully implemented a fully functional gym management REST API with 40+ endpoints
- Achieved proper 3NF database normalization across all 9 tables
- Implemented multi-step atomic transactions that correctly roll back on failure
- Advanced SQL analytics including window functions, set operations, and subqueries are fully operational

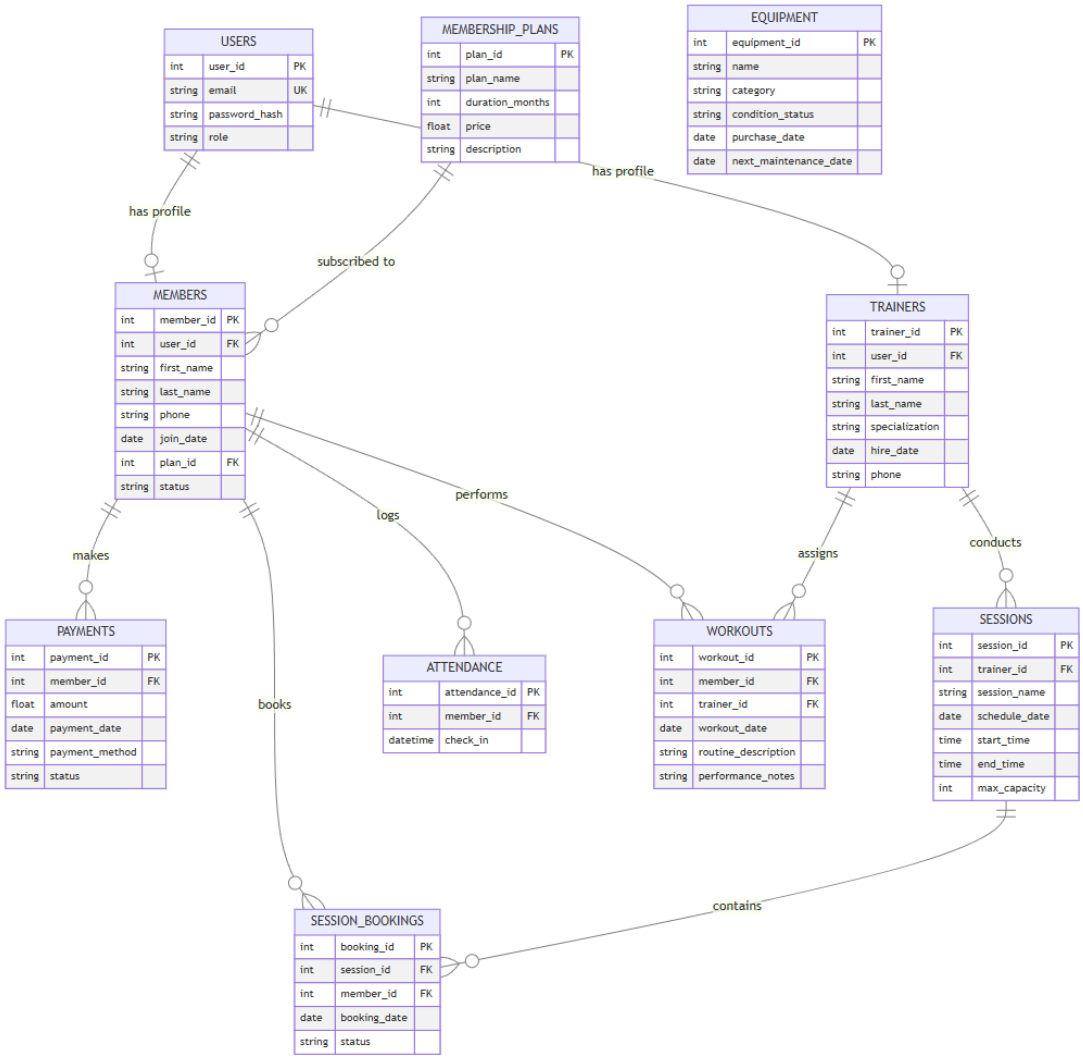
- Pydantic models enforce strict data validation across all API inputs and outputs
- Role-based access differentiation between admin, trainer, and member is enforced
- Equipment maintenance alert system proactively flags items needing attention

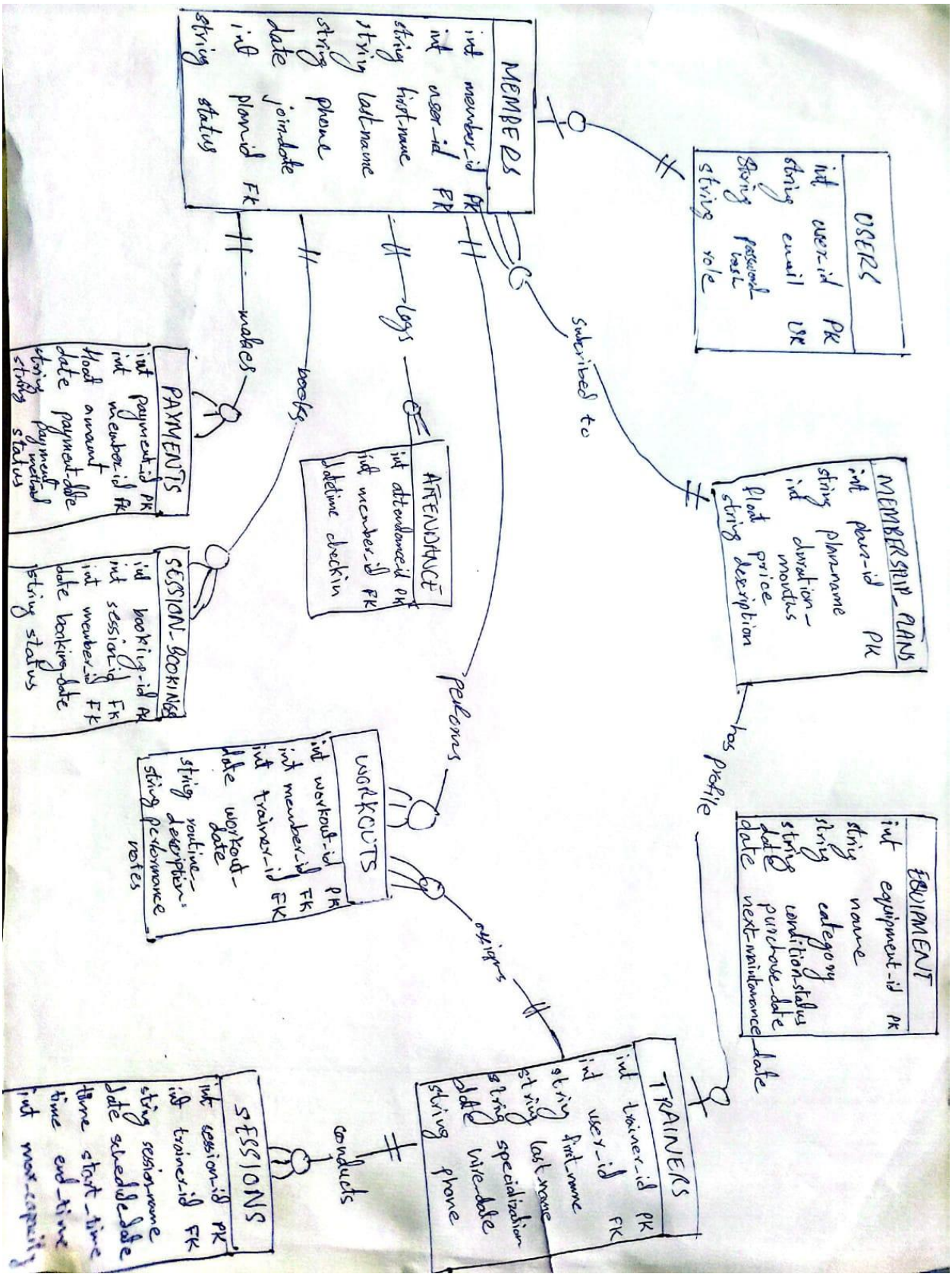
Issues Encountered

- PostgreSQL uses EXCEPT instead of MINUS for set difference operations - adjusted all queries accordingly
- Float division in fill rate percentage required casting to 100.0 to avoid integer truncation
- RETURNING clause syntax was needed for capturing auto-generated user_id in member/trainer registration transactions
- ILIKE was required for case-insensitive equipment condition matching in maintenance alerts
- Unique constraint errors on double booking required explicit try/except handling to return clean 400 errors

XII. Diagrams:

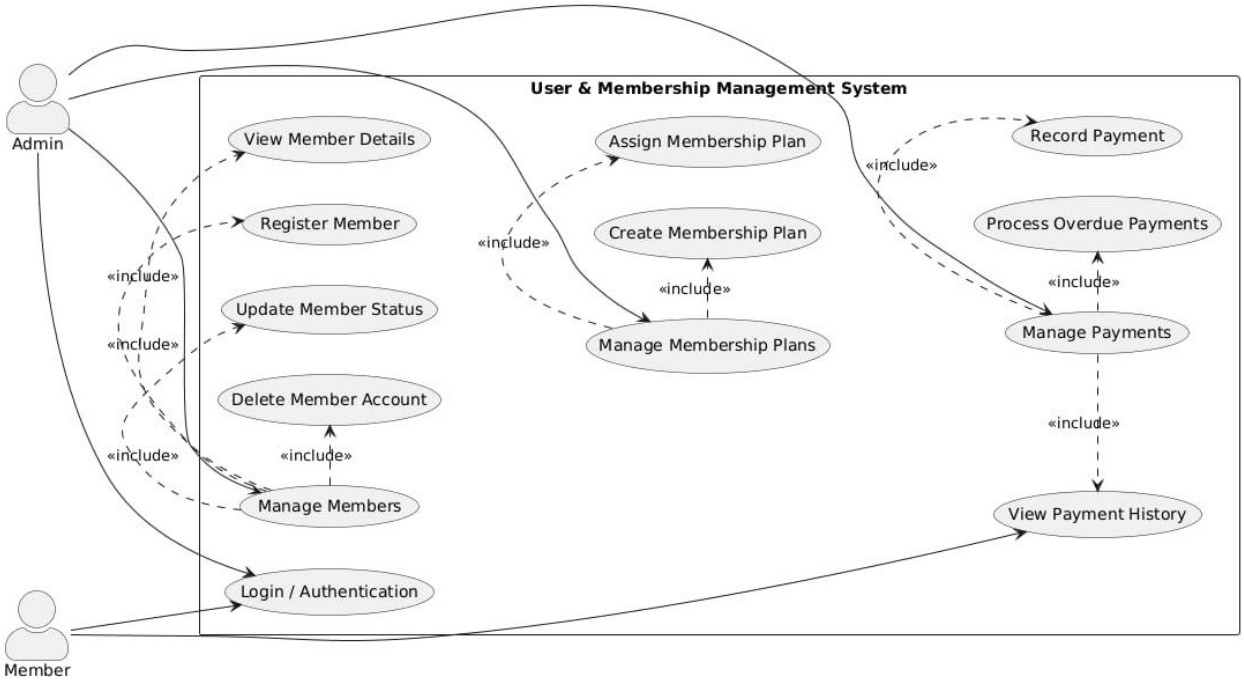
ER-Diagram:


USERS
int user_id PK
string email UK
string password_hash
string role
MEMBERSHIP_PLANS
int plan_id PK
string plan_name
int duration_months
float price
string description
EQUIPMENT
int equipment_id PK
string name
string category
string condition_status
date purchase_date
date next_maintenance_date
has profile
has profile
subscribed to
MEMBERS
int member_id PK
int user_id FK
string first_name
string last_name
string phone
date join_date
int plan_id FK
string status
TRAINERS
int trainer_id PK
int user_id FK
string first_name
string last_name
string specialization
date hire_date
string phone
performs
logs
makes
assigns
conducts
PAYMENTS
int payment_id PK
int member_id FK
float amount
date payment_date
string payment_method
string status
ATTENDANCE
int attendance_id PK
int member_id FK
datetime check_in
WORKOUTS
int workout_id PK
int member_id FK
int trainer_id FK
date workout_date
string routine_description
string performance_notes
SESSIONS
int session_id PK
int trainer_id FK
string session_name
date schedule_date
time start_time
time end_time
int max_capacity
books
contains
SESSION_BOOKINGS
int booking_id PK
int session_id FK
int member_id FK
date booking_date
string status

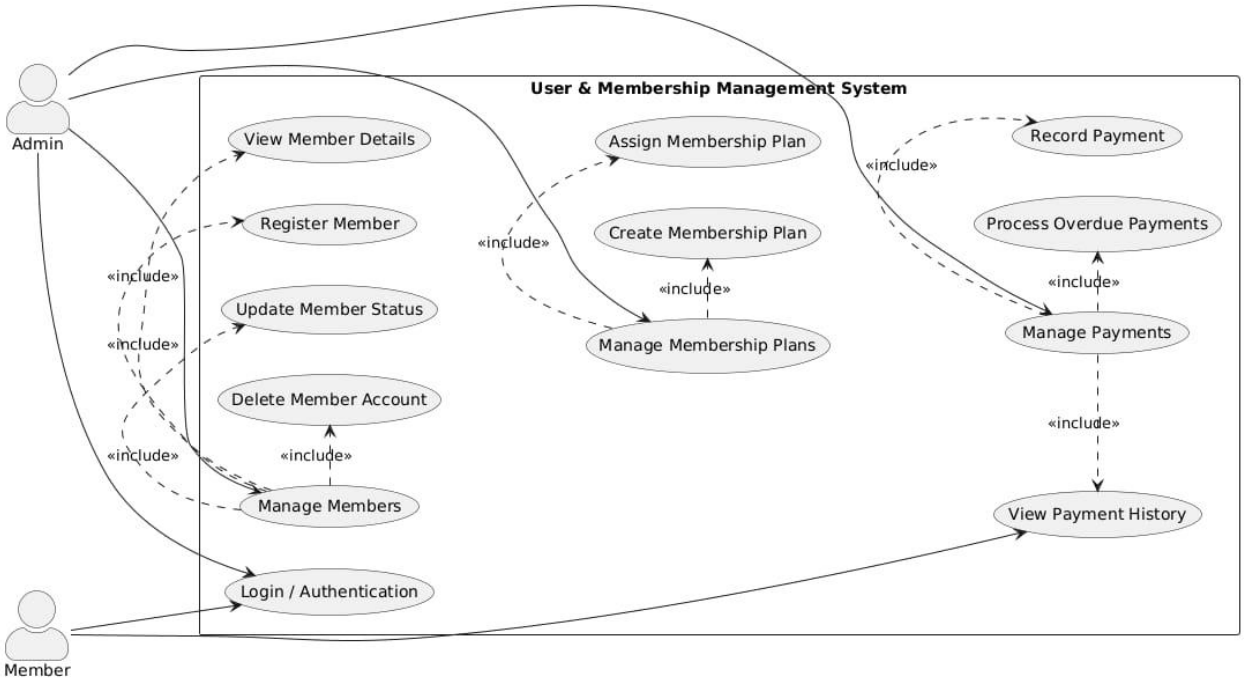

USERS
int user_id PK
string email UK
String password_hash
string role
MEMBERS
int member_id PK
int user_id FK
string first_name
string last_name
string phone
date join_date
int plan_id FK
string status
MEMBERSHIP_PLANS
int plan_id PK
string plan_name
int duration_months
float price
string description
EQUIPMENT
int equipment_id PK
string name
string category
string condition_status
date purchase_date
date next_maintenance_date
TRAINERS
int trainer_id PK
int user_id FK
string first_name
string last_name
string specialization
string hire_date
string phone
ATTENDANCE
int attendance_id PK
int member_id FK
datetime checkin
PAYMENTS
int payment_id PK
int member_id FK
float amount
date payment_date
string payment_method
string status
SESSION_BOOKINGS
int booking_id PK
int session_id FK
int member_id FK
date booking_date
string status
WORKOUTS
int workout_id PK
int member_id FK
int trainer_id FK
date workout_date
string routine_description
string performance_notes
SESSIONS
int session_id PK
int trainer_id FK
string session_name
date scheduled_date
time start_time
time end_time
int max_capacity
subscribed to
has profile
logs
books
makes
performs
assigns
conducts

Use-Case Diagrams:

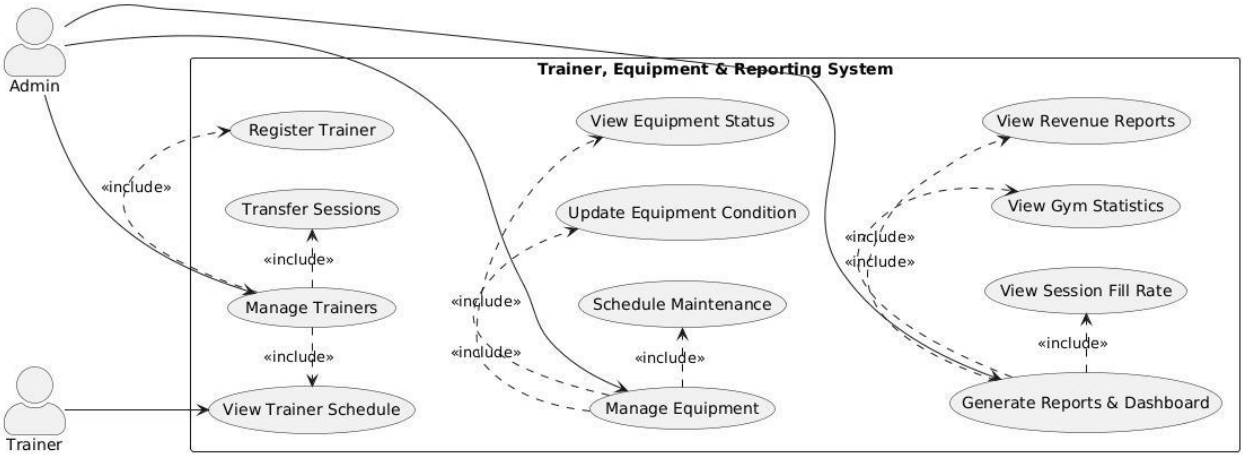
1. User & Membership Management Use Case Diagram



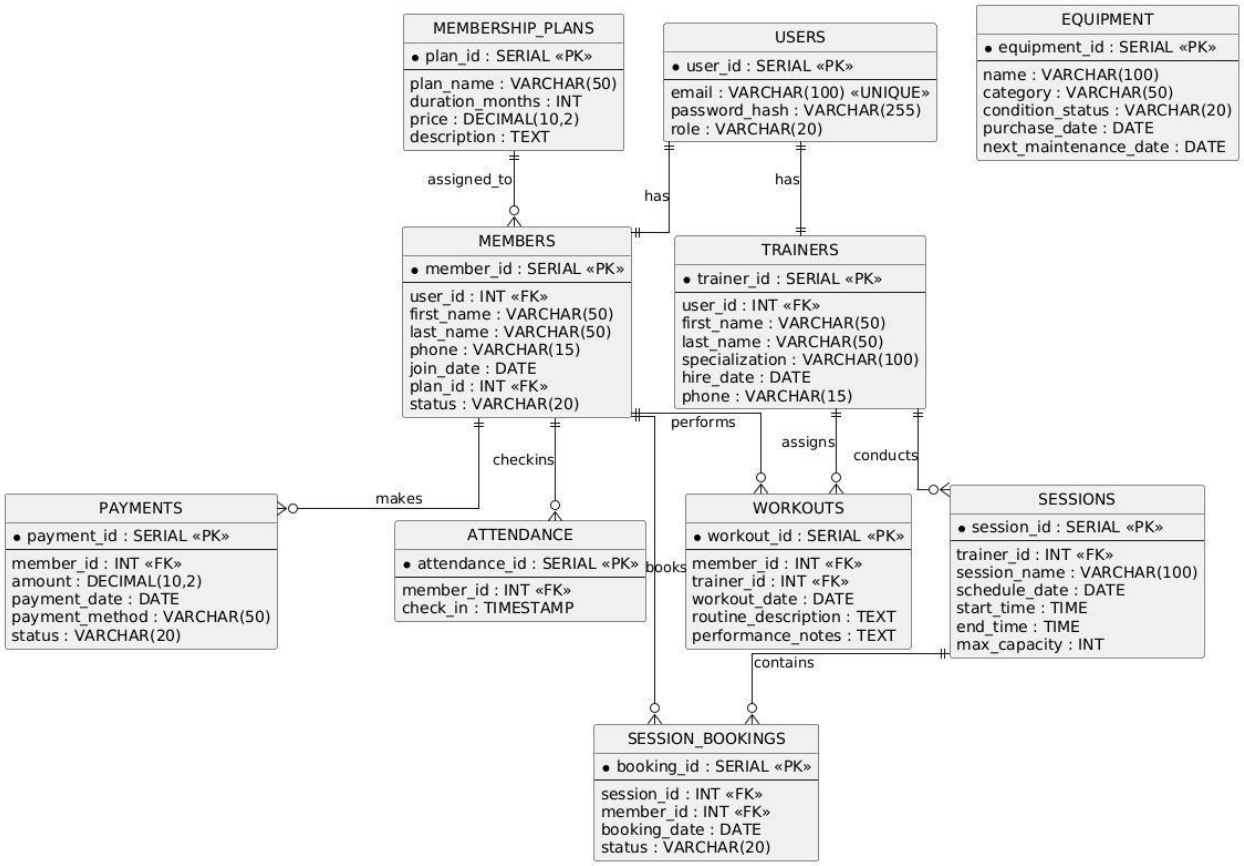
2.. Session, Workout & Attendance Management Use Case Diagram:



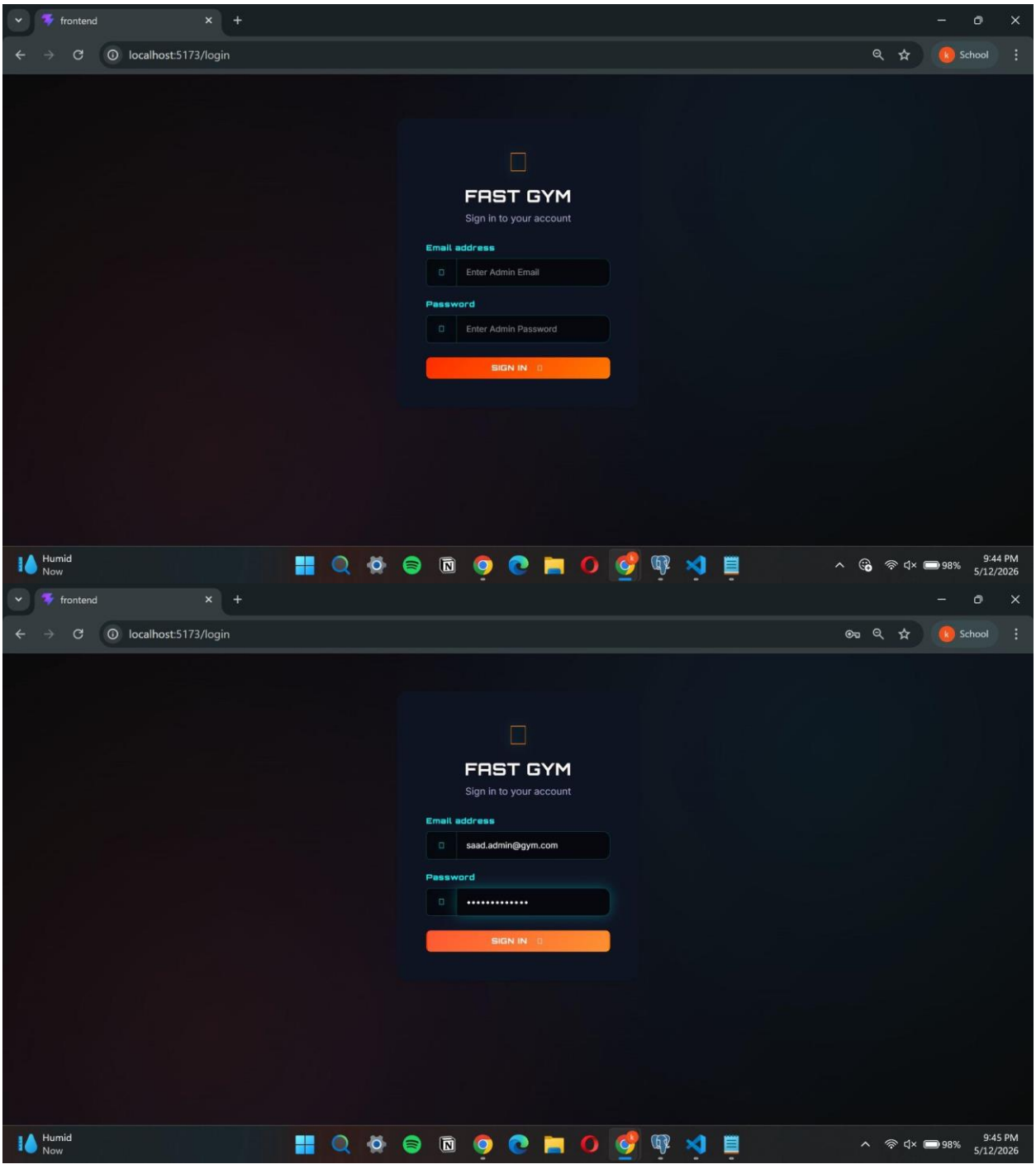
3. Trainer, Equipment & Reports Management Use Case Diagram:

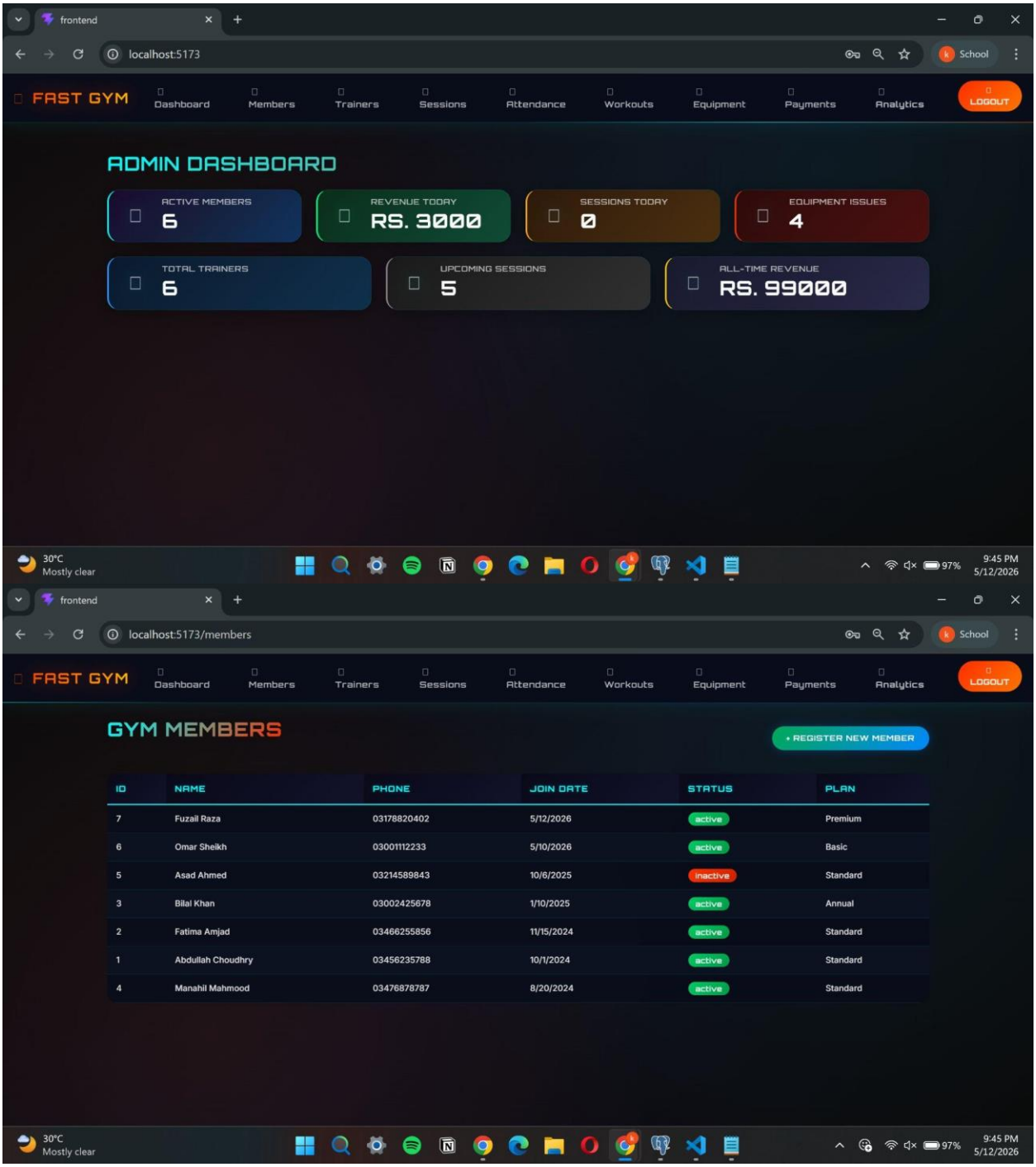


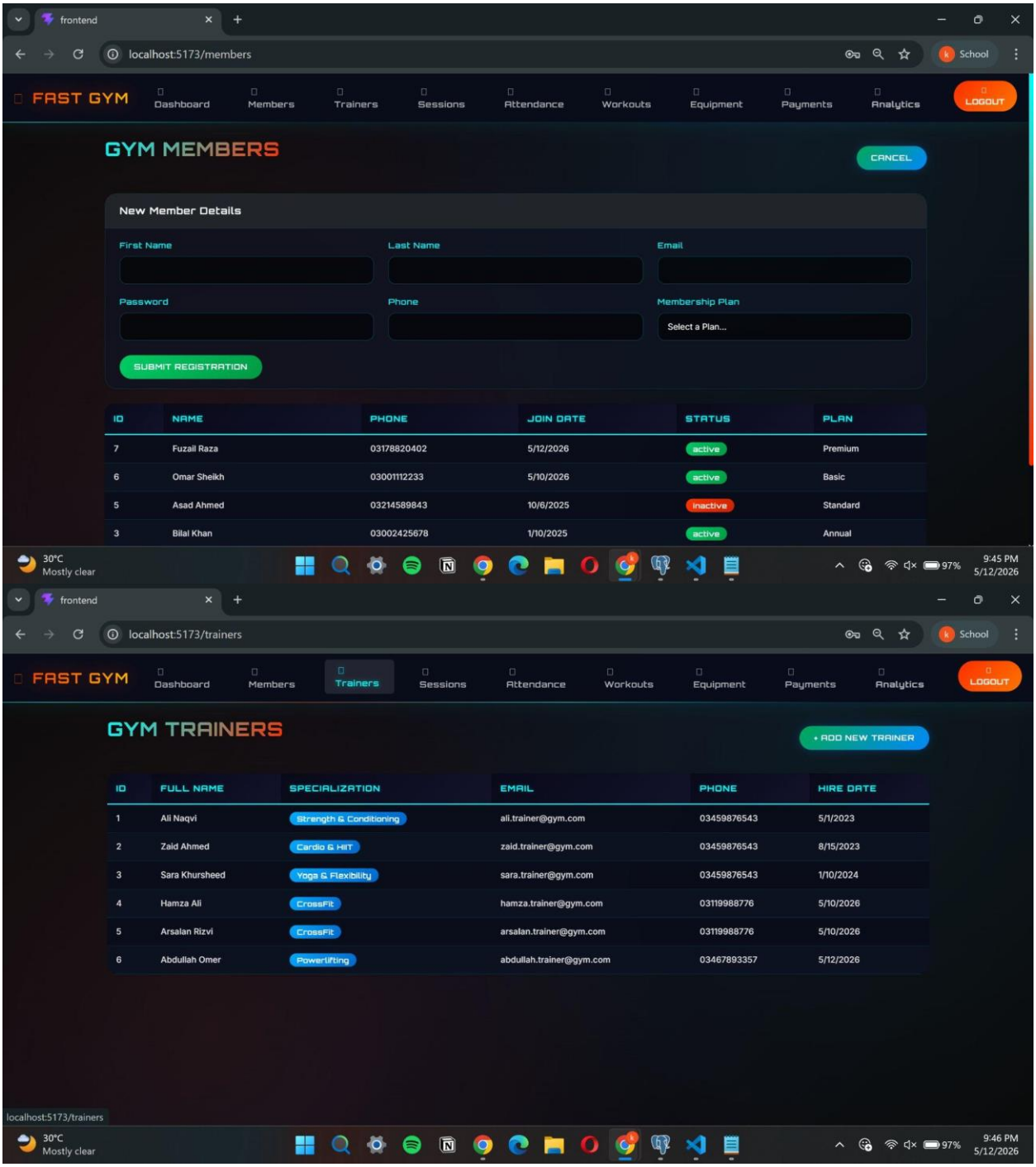
Schema Relationship Diagram:

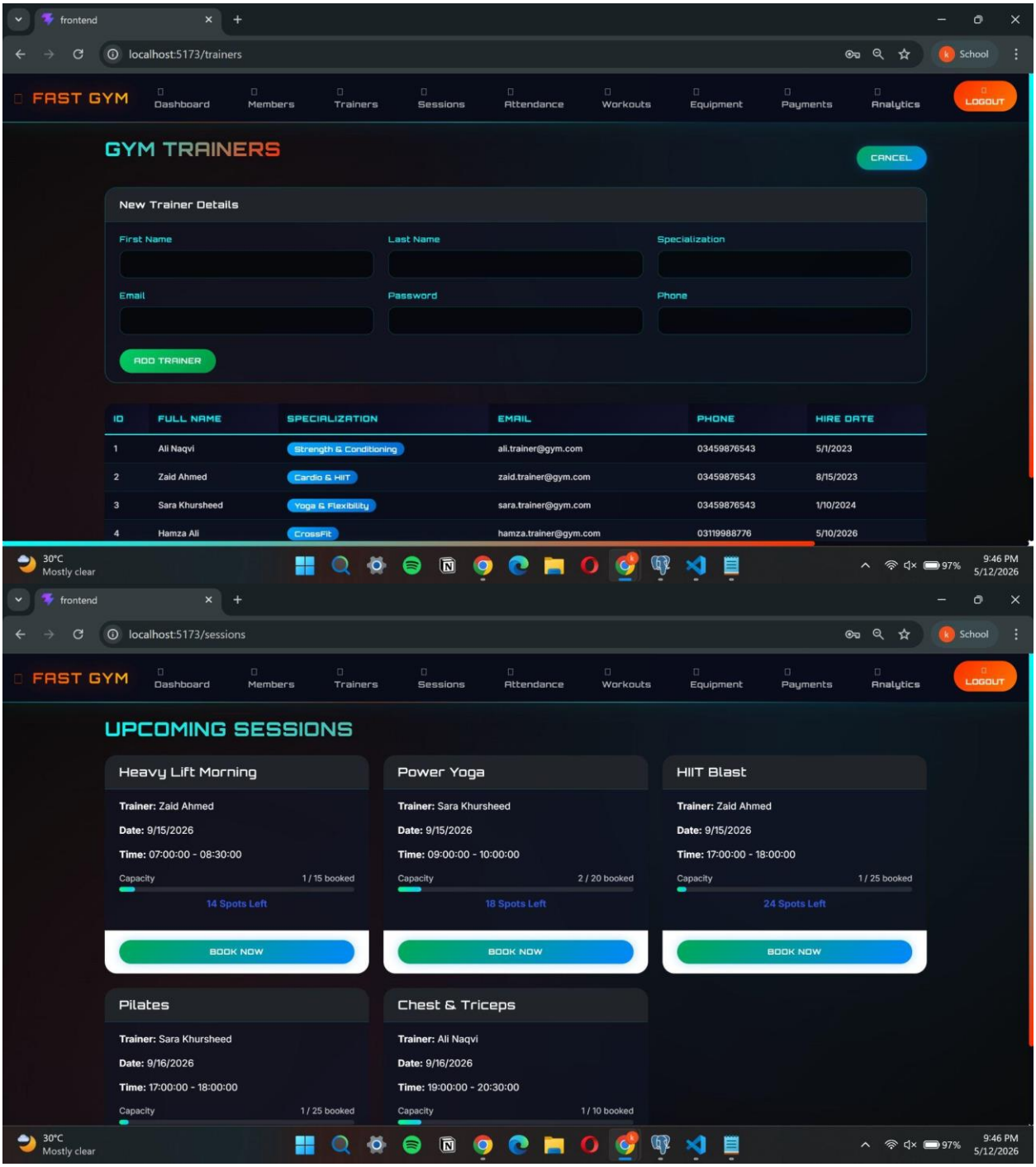


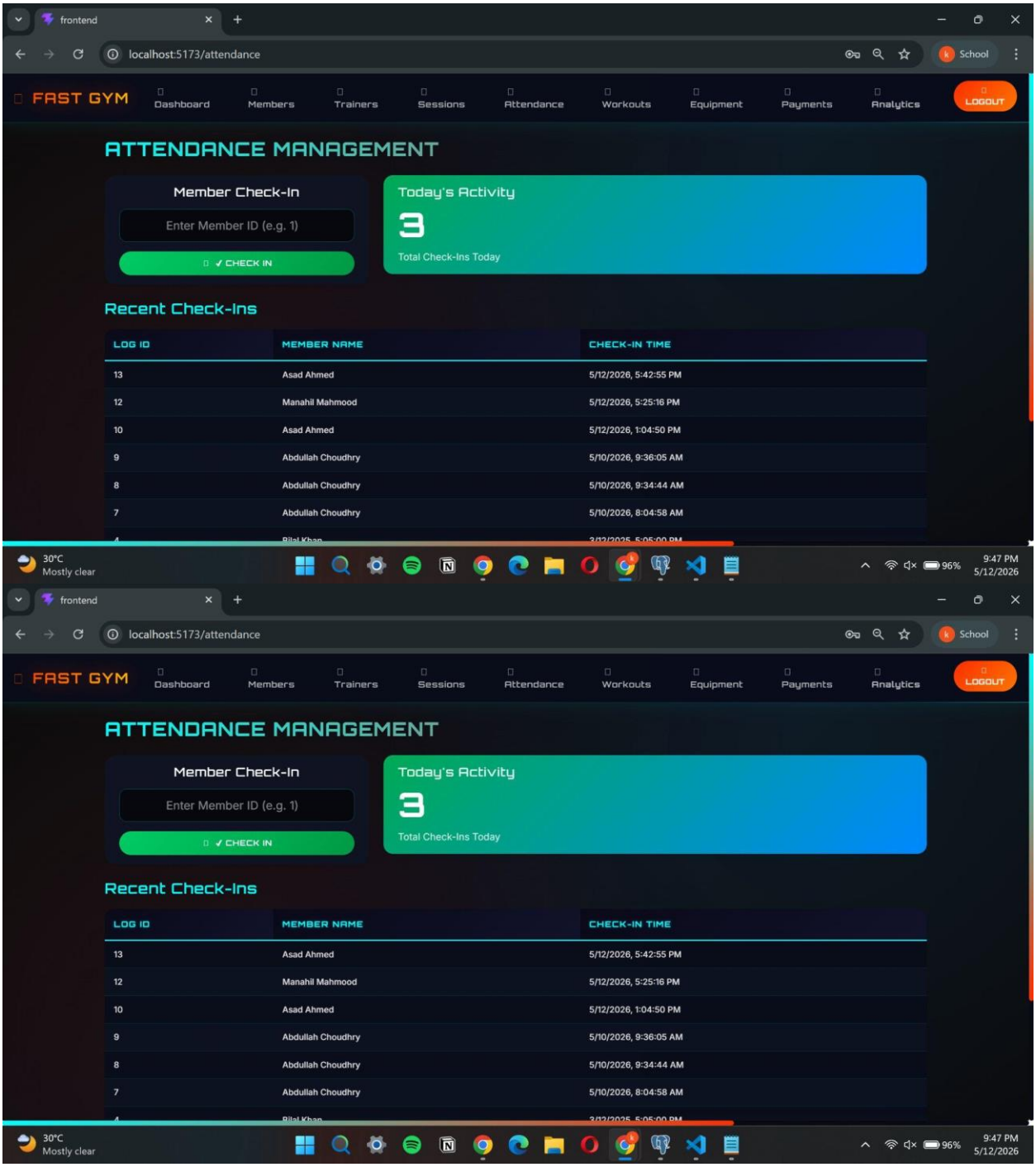
ScreenShots:

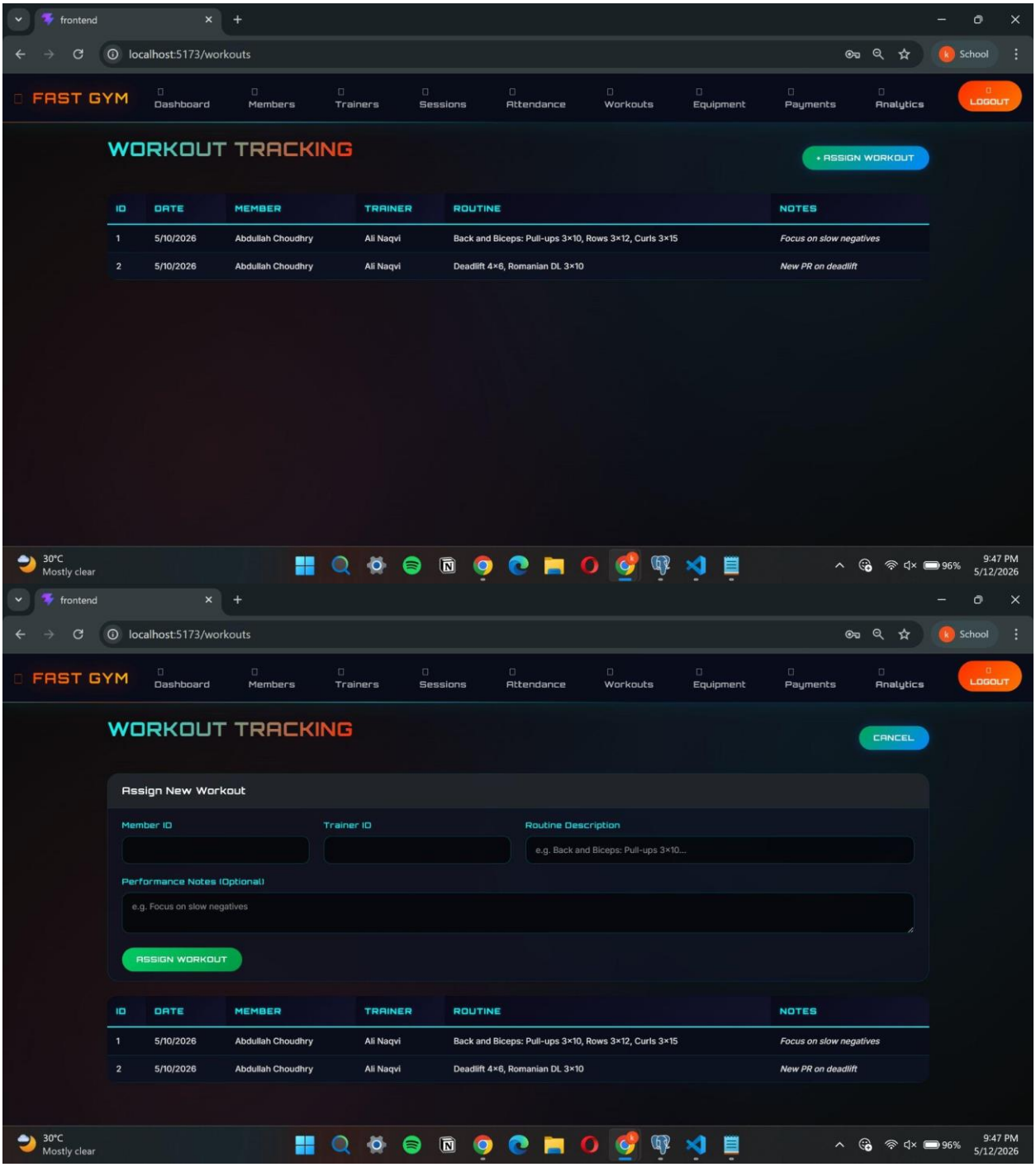


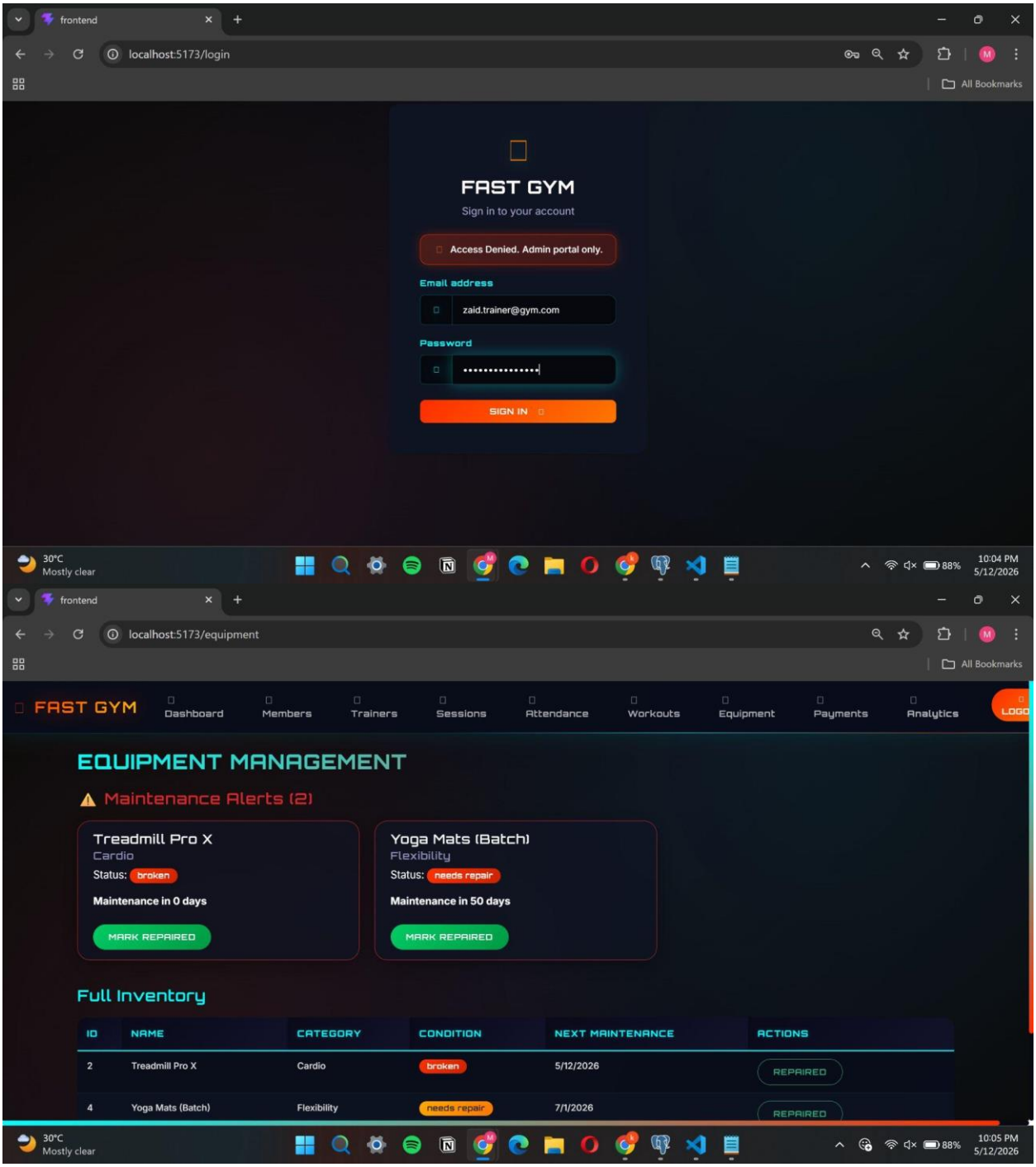

frontend
localhost:5173
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
ADMIN DASHBOARD
ACTIVE MEMBERS
6
REVENUE TODAY
RS. 3000
SESSIONS TODAY
0
EQUIPMENT ISSUES
4
TOTAL TRAINERS
6
UPCOMING SESSIONS
5
ALL-TIME REVENUE
RS. 99000
30°C
Mostly clear
97%
9:45 PM
5/12/2026
frontend
localhost:5173/members
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
GYM MEMBERS
+ REGISTER NEW MEMBER
ID
NAME
PHONE
JOIN DATE
STATUS
PLAN
7
Fuzail Raza
03178820402
5/12/2026
active
Premium
6
Omar Sheikh
03001112233
5/10/2026
active
Basic
5
Asad Ahmed
03214589843
10/6/2025
inactive
Standard
3
Bilal Khan
03002425678
1/10/2025
active
Annual
2
Fatima Amjad
03466255856
11/15/2024
active
Standard
1
Abdullah Choudhry
03456235788
10/1/2024
active
Standard
4
Manahil Mahmood
03476878787
8/20/2024
active
Standard
30°C
Mostly clear
97%
9:45 PM
5/12/2026

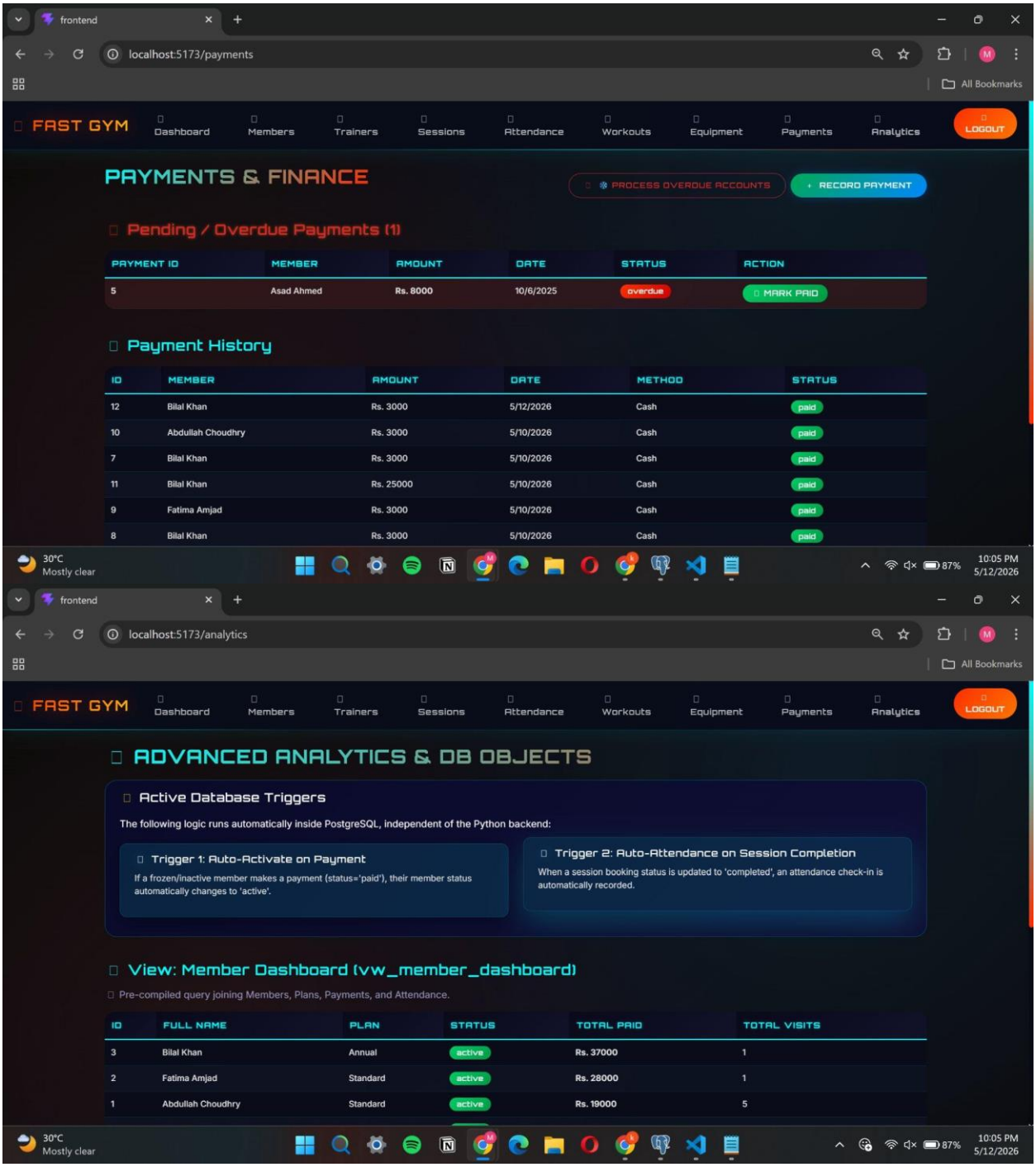

frontend
localhost:5173/members
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
GYM MEMBERS
CANCEL
New Member Details
First Name
Last Name
Email
Password
Phone
Membership Plan
Select a Plan...
SUBMIT REGISTRATION
ID NAME PHONE JOIN DATE STATUS PLAN
7 Fuzail Raza 03178820402 5/12/2026 active Premium
6 Omar Sheikh 03001112233 5/10/2026 active Basic
5 Asad Ahmed 03214589843 10/6/2025 inactive Standard
3 Bilal Khan 03002425678 1/10/2025 active Annual
30°C Mostly clear
97%
9:45 PM 5/12/2026
frontend
localhost:5173/trainers
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
GYM TRAINERS
+ ADD NEW TRAINER
ID FULL NAME SPECIALIZATION EMAIL PHONE HIRE DATE
1 Ali Naqvi Strength & Conditioning ali.trainer@gym.com 03459876543 5/1/2023
2 Zaid Ahmed Cardio & HIIT zaid.trainer@gym.com 03459876543 8/15/2023
3 Sara Khursheed Yoga & Flexibility sara.trainer@gym.com 03459876543 1/10/2024
4 Hamza Ali CrossFit hamza.trainer@gym.com 03119988776 5/10/2026
5 Arsalan Rizvi CrossFit arsalan.trainer@gym.com 03119988776 5/10/2026
6 Abdullah Omer Powerlifting abdullah.trainer@gym.com 03467893357 5/12/2026
localhost:5173/trainers
30°C Mostly clear
97%
9:46 PM 5/12/2026

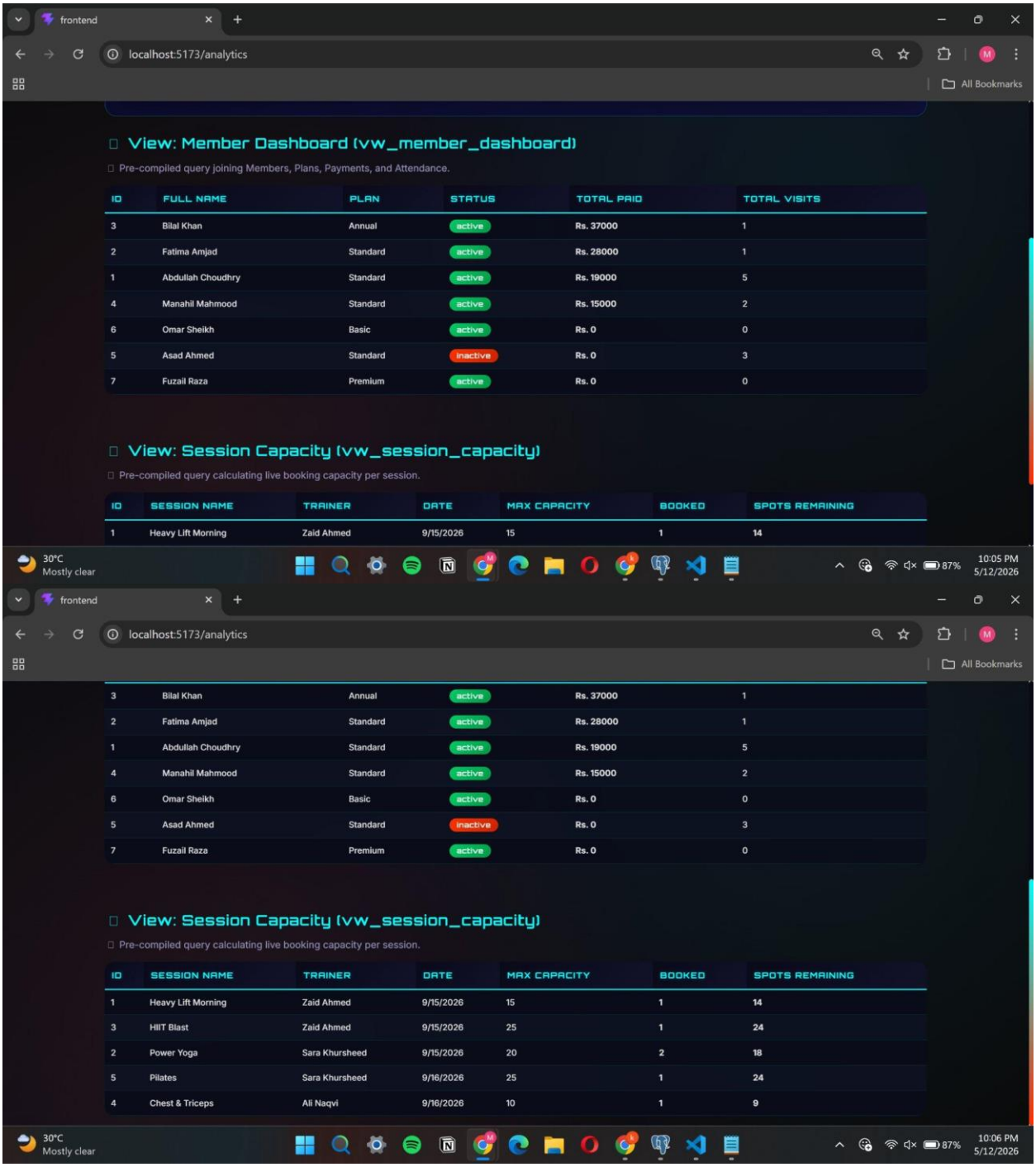

frontend
localhost:5173/trainers
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
GYM TRAINERS
CANCEL
New Trainer Details
First Name
Last Name
Specialization
Email
Password
Phone
ADD TRAINER
ID
FULL NAME
SPECIALIZATION
EMAIL
PHONE
HIRE DATE
1
Ali Naqvi
Strength & Conditioning
ali.trainer@gym.com
03459876543
5/1/2023
2
Zaid Ahmed
Cardio & HIIT
zaid.trainer@gym.com
03459876543
8/15/2023
3
Sara Khursheed
Yoga & Flexibility
sara.trainer@gym.com
03459876543
1/10/2024
4
Hamza Ali
CrossFit
hamza.trainer@gym.com
03119988776
5/10/2026
30°C
Mostly clear
97%
9:46 PM
5/12/2026
frontend
localhost:5173/sessions
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
UPCOMING SESSIONS
Heavy Lift Morning
Trainer: Zaid Ahmed
Date: 9/15/2026
Time: 07:00:00 - 08:30:00
Capacity
1 / 15 booked
14 Spots Left
BOOK NOW
Power Yoga
Trainer: Sara Khursheed
Date: 9/15/2026
Time: 09:00:00 - 10:00:00
Capacity
2 / 20 booked
18 Spots Left
BOOK NOW
HIIT Blast
Trainer: Zaid Ahmed
Date: 9/15/2026
Time: 17:00:00 - 18:00:00
Capacity
1 / 25 booked
24 Spots Left
BOOK NOW
Pilates
Trainer: Sara Khursheed
Date: 9/16/2026
Time: 17:00:00 - 18:00:00
Capacity
1 / 25 booked
Chest & Triceps
Trainer: Ali Naqvi
Date: 9/16/2026
Time: 19:00:00 - 20:30:00
Capacity
1 / 10 booked
30°C
Mostly clear
97%
9:46 PM
5/12/2026

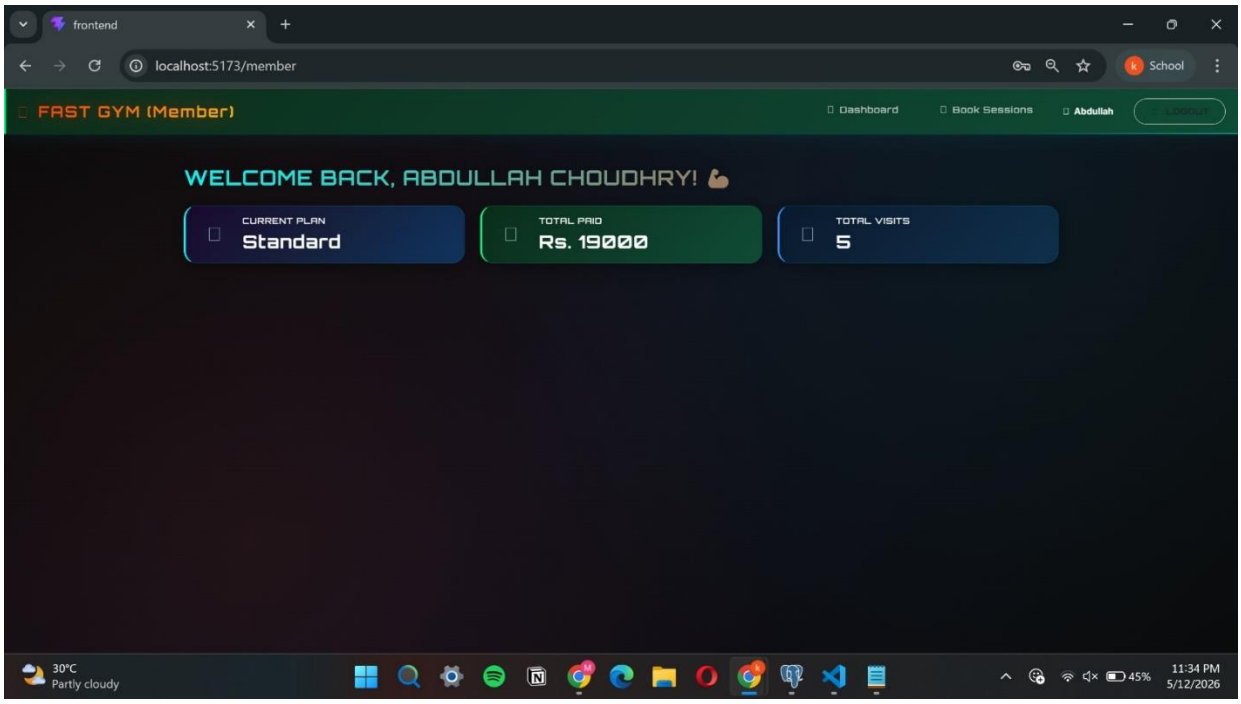

frontend
localhost:5173/attendance
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
ATTENDANCE MANAGEMENT
Member Check-In
Enter Member ID (e.g. 1)
✓ CHECK IN
Today's Activity
3
Total Check-Ins Today
Recent Check-Ins
LOG ID
MEMBER NAME
CHECK-IN TIME
13 Asad Ahmed 5/12/2026, 5:42:55 PM
12 Manahil Mahmood 5/12/2026, 5:25:16 PM
10 Asad Ahmed 5/12/2026, 1:04:50 PM
9 Abdullah Choudhry 5/10/2026, 9:36:05 AM
8 Abdullah Choudhry 5/10/2026, 9:34:44 AM
7 Abdullah Choudhry 5/10/2026, 8:04:58 AM
30°C
Mostly clear
96%
9:47 PM
5/12/2026

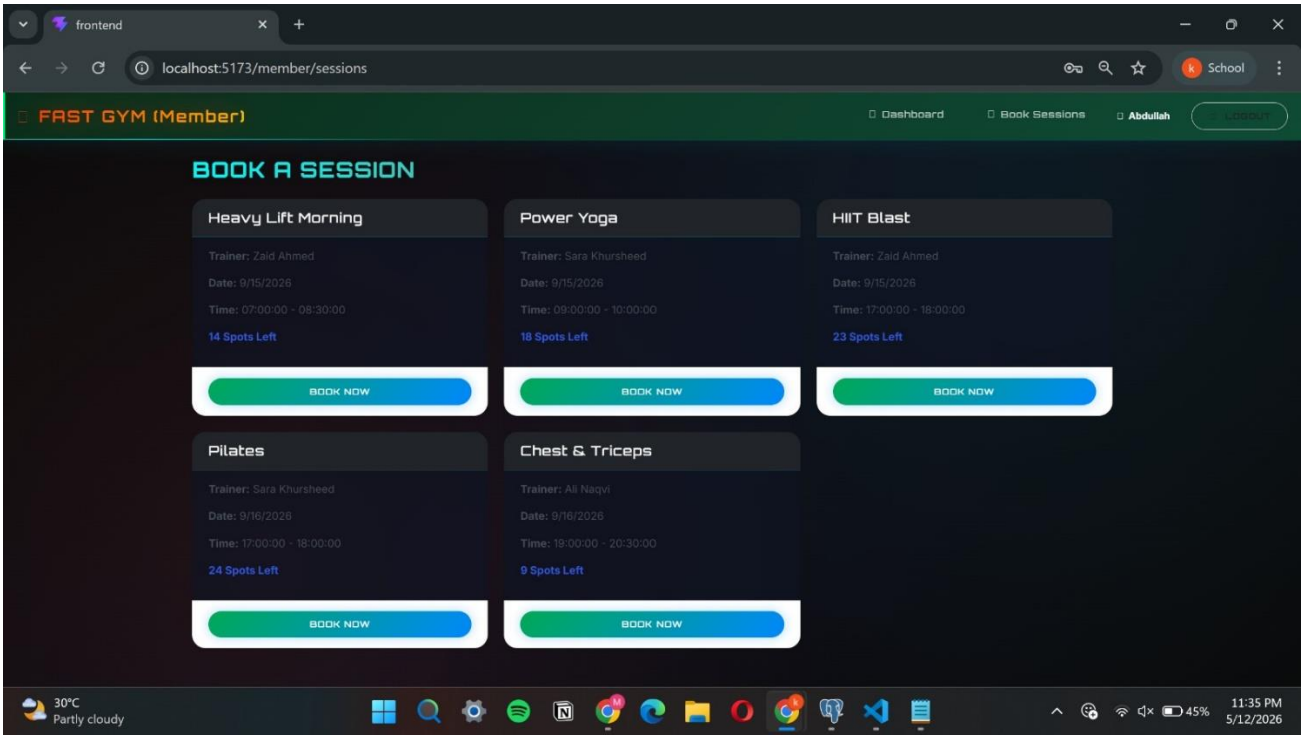

frontend
localhost:5173/workouts
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
WORKOUT TRACKING
+ ASSIGN WORKOUT
ID
DATE
MEMBER
TRAINER
ROUTINE
NOTES
1
5/10/2026
Abdullah Choudhry
Ali Naqvi
Back and Biceps: Pull-ups 3×10, Rows 3×12, Curls 3×15
Focus on slow negatives
2
5/10/2026
Abdullah Choudhry
Ali Naqvi
Deadlift 4×6, Romanian DL 3×10
New PR on deadlift
30°C
Mostly clear
96%
9:47 PM
5/12/2026
frontend
localhost:5173/workouts
School
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
WORKOUT TRACKING
CANCEL
Assign New Workout
Member ID
Trainer ID
Routine Description
e.g. Back and Biceps: Pull-ups 3×10...
Performance Notes (Optional)
e.g. Focus on slow negatives
ASSIGN WORKOUT
ID
DATE
MEMBER
TRAINER
ROUTINE
NOTES
1
5/10/2026
Abdullah Choudhry
Ali Naqvi
Back and Biceps: Pull-ups 3×10, Rows 3×12, Curls 3×15
Focus on slow negatives
2
5/10/2026
Abdullah Choudhry
Ali Naqvi
Deadlift 4×6, Romanian DL 3×10
New PR on deadlift
30°C
Mostly clear
96%
9:47 PM
5/12/2026

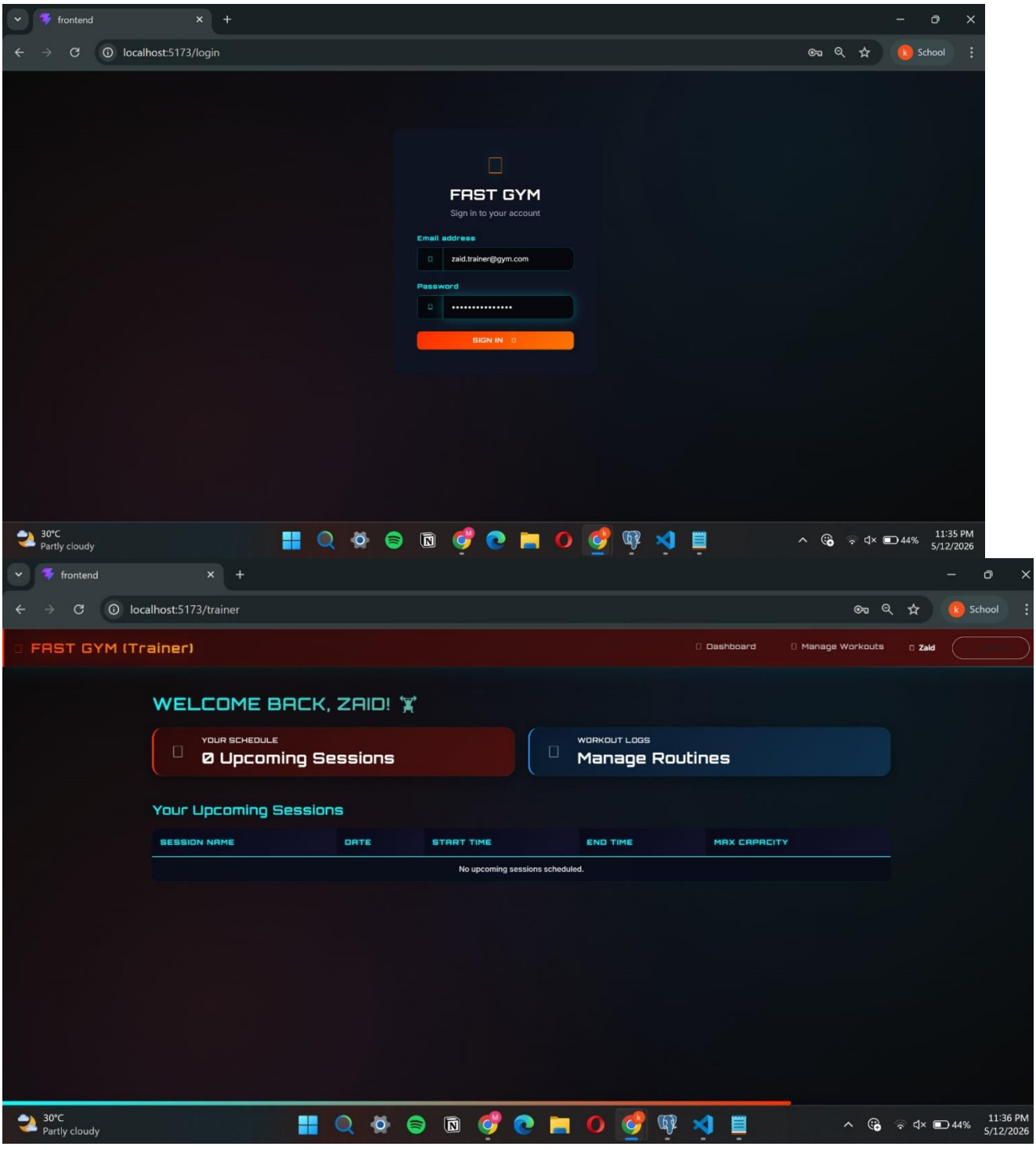

frontend
localhost:5173/login
All Bookmarks
FAST GYM
Sign in to your account
Access Denied. Admin portal only.
Email address
zaid.trainer@gym.com
Password
SIGN IN
30°C
Mostly clear
88%
10:04 PM
5/12/2026
frontend
localhost:5173/equipment
All Bookmarks
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGO
EQUIPMENT MANAGEMENT
Maintenance Alerts (2)
Treadmill Pro X
Cardio
Status: broken
Maintenance in 0 days
MARK REPAIRED
Yoga Mats (Batch)
Flexibility
Status: needs repair
Maintenance in 50 days
MARK REPAIRED
Full Inventory
ID
NAME
CATEGORY
CONDITION
NEXT MAINTENANCE
ACTIONS
2
Treadmill Pro X
Cardio
broken
5/12/2026
REPAIRED
4
Yoga Mats (Batch)
Flexibility
needs repair
7/1/2026
REPAIRED
30°C
Mostly clear
88%
10:05 PM
5/12/2026

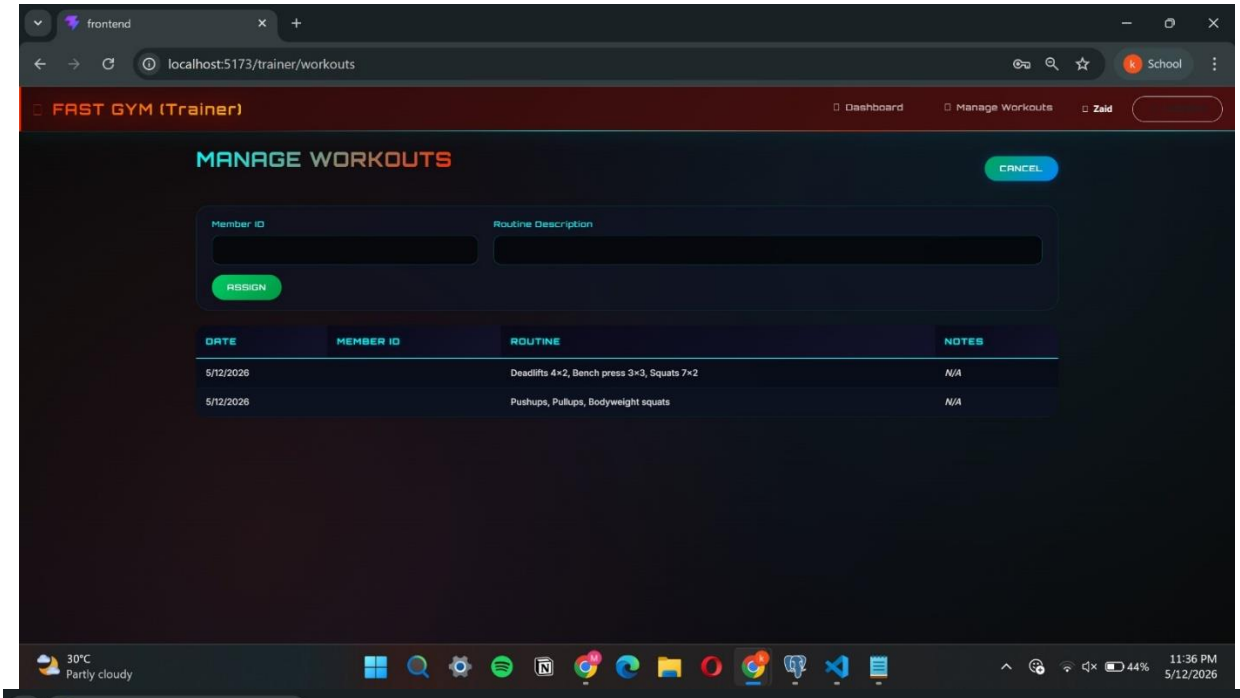

frontend
localhost:5173/payments
All Bookmarks
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
PAYMENTS & FINANCE
PROCESS OVERDUE ACCOUNTS
RECORD PAYMENT
Pending / Overdue Payments (1)
PAYMENT ID
MEMBER
AMOUNT
DATE
STATUS
ACTION
5
Asad Ahmed
Rs. 8000
10/6/2025
overdue
MARK PAID
Payment History
ID
MEMBER
AMOUNT
DATE
METHOD
STATUS
12
Bilal Khan
Rs. 3000
5/12/2026
Cash
paid
10
Abdullah Choudhry
Rs. 3000
5/10/2026
Cash
paid
7
Bilal Khan
Rs. 3000
5/10/2026
Cash
paid
11
Bilal Khan
Rs. 25000
5/10/2026
Cash
paid
9
Fatima Amjad
Rs. 3000
5/10/2026
Cash
paid
8
Bilal Khan
Rs. 3000
5/10/2026
Cash
paid
30°C
Mostly clear
87%
10:05 PM
5/12/2026
frontend
localhost:5173/analytics
All Bookmarks
FAST GYM
Dashboard
Members
Trainers
Sessions
Attendance
Workouts
Equipment
Payments
Analytics
LOGOUT
ADVANCED ANALYTICS & DB OBJECTS
Active Database Triggers
The following logic runs automatically inside PostgreSQL, independent of the Python backend:
Trigger 1: Auto-Activate on Payment
If a frozen/inactive member makes a payment (status='paid'), their member status automatically changes to 'active'.
Trigger 2: Auto-Attendance on Session Completion
When a session booking status is updated to 'completed', an attendance check-in is automatically recorded.
View: Member Dashboard (vw_member_dashboard)
Pre-compiled query joining Members, Plans, Payments, and Attendance.
ID
FULL NAME
PLAN
STATUS
TOTAL PAID
TOTAL VISITS
3
Bilal Khan
Annual
active
Rs. 37000
1
2
Fatima Amjad
Standard
active
Rs. 28000
1
1
Abdullah Choudhry
Standard
active
Rs. 19000
5
30°C
Mostly clear
87%
10:05 PM
5/12/2026

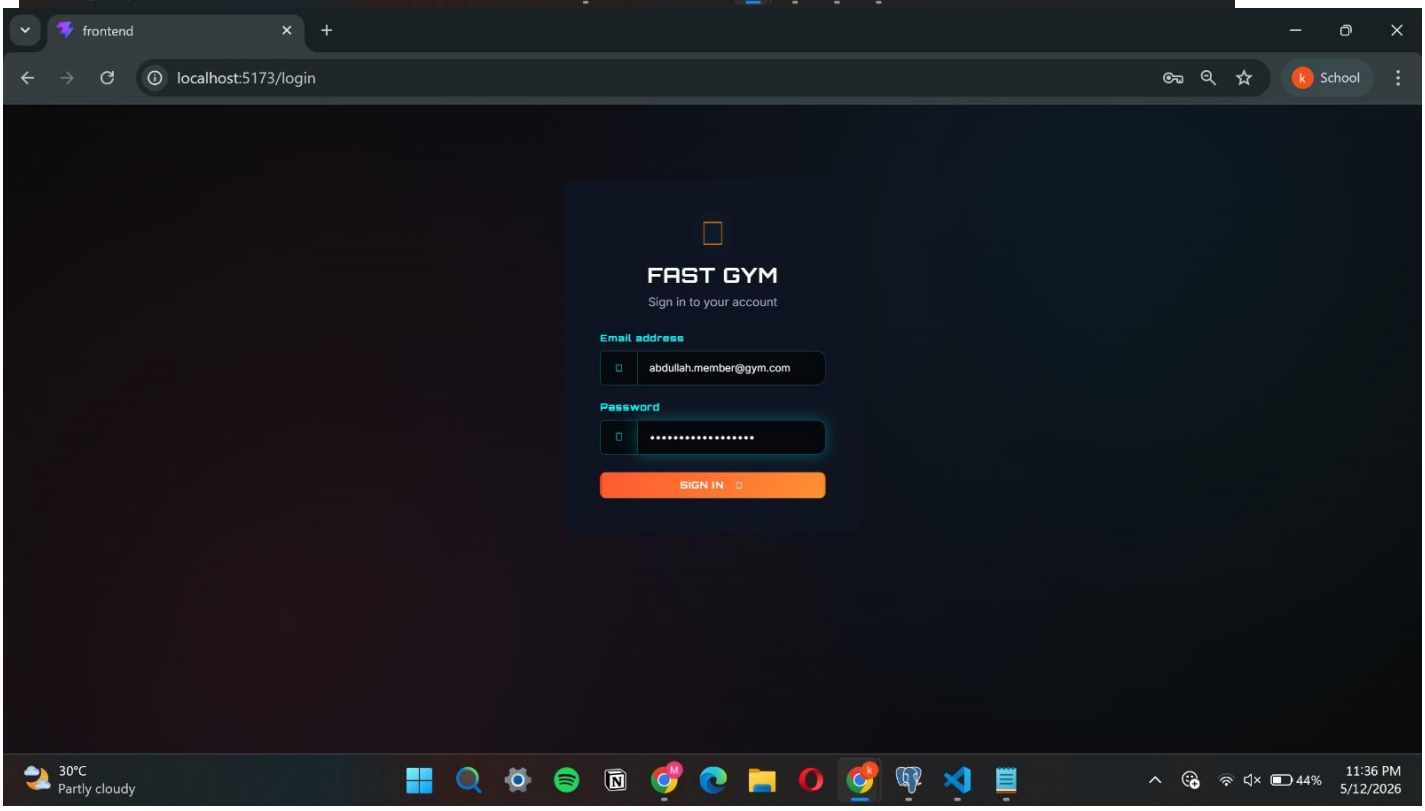

frontend
localhost:5173/analytics
All Bookmarks
View: Member Dashboard (vw_member_dashboard)
Pre-compiled query joining Members, Plans, Payments, and Attendance.
ID FULL NAME PLAN STATUS TOTAL PAID TOTAL VISITS
3 Bilal Khan Annual active Rs. 37000 1
2 Fatima Amjad Standard active Rs. 28000 1
1 Abdullah Choudhry Standard active Rs. 19000 5
4 Manahil Mahmood Standard active Rs. 15000 2
6 Omar Sheikh Basic active Rs. 0 0
5 Asad Ahmed Standard inactive Rs. 0 3
7 Fuzail Raza Premium active Rs. 0 0
View: Session Capacity (vw_session_capacity)
Pre-compiled query calculating live booking capacity per session.
ID SESSION NAME TRAINER DATE MAX CAPACITY BOOKED SPOTS REMAINING
1 Heavy Lift Morning Zaid Ahmed 9/15/2026 15 1 14
30°C
Mostly clear
87%
10:05 PM
5/12/2026
frontend
localhost:5173/analytics
All Bookmarks
3 Bilal Khan Annual active Rs. 37000 1
2 Fatima Amjad Standard active Rs. 28000 1
1 Abdullah Choudhry Standard active Rs. 19000 5
4 Manahil Mahmood Standard active Rs. 15000 2
6 Omar Sheikh Basic active Rs. 0 0
5 Asad Ahmed Standard inactive Rs. 0 3
7 Fuzail Raza Premium active Rs. 0 0
View: Session Capacity (vw_session_capacity)
Pre-compiled query calculating live booking capacity per session.
ID SESSION NAME TRAINER DATE MAX CAPACITY BOOKED SPOTS REMAINING
1 Heavy Lift Morning Zaid Ahmed 9/15/2026 15 1 14
3 HIIT Blast Zaid Ahmed 9/15/2026 25 1 24
2 Power Yoga Sara Khursheed 9/15/2026 20 2 18
5 Pilates Sara Khursheed 9/16/2026 25 1 24
4 Chest & Triceps Ali Naqvi 9/16/2026 10 1 9
30°C
Mostly clear
87%
10:06 PM
5/12/2026

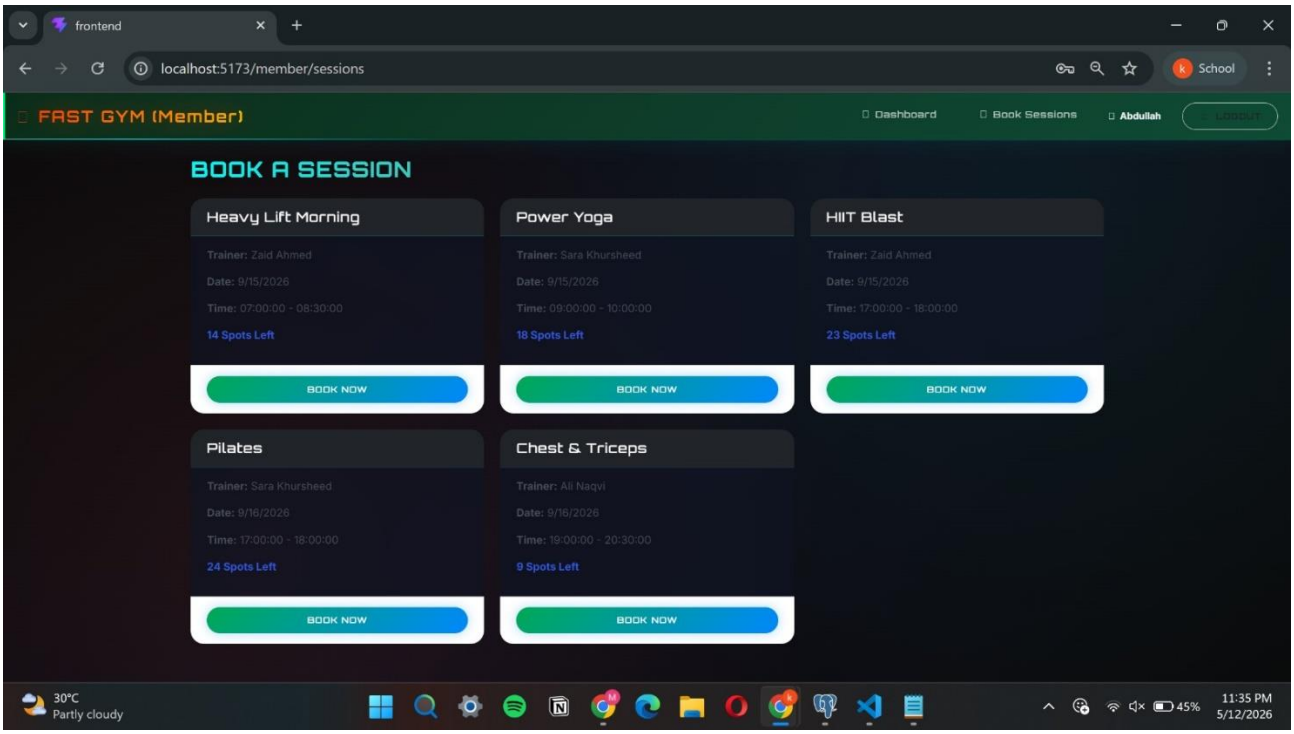

frontend
localhost:5173/member
School
FAST GYM (Member)
Dashboard
Book Sessions
Abdullah
LOGOUT
WELCOME BACK, ABDULLAH CHOUDHRY!
CURRENT PLAN
Standard
TOTAL PAID
Rs. 19000
TOTAL VISITS
5
30°C
Partly cloudy
45%
11:34 PM
5/12/2026


frontend
localhost:5173/member/sessions
School
FAST GYM (Member)
Dashboard
Book Sessions
Abdullah
LOGOUT
BOOK A SESSION
Heavy Lift Morning
Trainer: Zaid Ahmed
Date: 9/15/2026
Time: 07:00:00 - 08:30:00
14 Spots Left
BOOK NOW
Power Yoga
Trainer: Sara Khursheed
Date: 9/15/2026
Time: 09:00:00 - 10:00:00
18 Spots Left
BOOK NOW
HIIT Blast
Trainer: Zaid Ahmed
Date: 9/15/2026
Time: 17:00:00 - 18:00:00
23 Spots Left
BOOK NOW
Pilates
Trainer: Sara Khursheed
Date: 9/16/2026
Time: 17:00:00 - 18:00:00
24 Spots Left
BOOK NOW
Chest & Triceps
Trainer: Ali Naqvi
Date: 9/16/2026
Time: 19:00:00 - 20:30:00
9 Spots Left
BOOK NOW
30°C
Partly cloudy
45%
11:35 PM
5/12/2026


frontend
localhost:5173/login
School
FAST GYM
Sign in to your account
Email address
zaid.trainer@gym.com
Password
SIGN IN
30°C
Partly cloudy
44%
11:35 PM
5/12/2026
frontend
localhost:5173/trainer
School
FAST GYM (Trainer)
Dashboard
Manage Workouts
Zaid
WELCOME BACK, ZAID!
YOUR SCHEDULE
0 Upcoming Sessions
WORKOUT LOGS
Manage Routines
Your Upcoming Sessions
SESSION NAME
DATE
START TIME
END TIME
MAX CAPACITY
No upcoming sessions scheduled.
30°C
Partly cloudy
44%
11:36 PM
5/12/2026


frontend
localhost:5173/trainer/workouts
School
FAST GYM (Trainer)
Dashboard
Manage Workouts
Zaid
MANAGE WORKOUTS
CANCEL
Member ID
Routine Description
ASSIGN
DATE
MEMBER ID
ROUTINE
NOTES
5/12/2026
Deadlifts 4×2, Bench press 3×3, Squats 7×2
N/A
5/12/2026
Pushups, Pullups, Bodyweight squats
N/A
30°C
Partly cloudy
44%
11:36 PM
5/12/2026


frontend
localhost:5173/login
School
FAST GYM
Sign in to your account
Email address
abdullah.member@gym.com
Password
SIGN IN
30°C
Partly cloudy
44%
11:36 PM
5/12/2026



XIII. Conclusion

The Gym Membership Management System successfully demonstrates a well-designed, production-quality database-driven application. The PostgreSQL schema efficiently models real-world gym operations through nine normalized tables with properly defined relationships, constraints, and cascade behaviors.

The FastAPI backend exposes a clean, documented REST API that covers all core gym operations while maintaining data integrity through transaction management. Advanced database features - including window functions for attendance ranking, UNION/INTERSECT/EXCEPT set operations for member activity analysis, and running revenue totals - demonstrate practical application of DBS concepts beyond basic CRUD operations.

The application proves particularly effective in:

- Centralizing member, trainer, session, and payment management in a single system
- Automating overdue payment detection and account freezing through batch transactions
- Providing real-time analytics on revenue, session capacity, and member engagement
- Ensuring data consistency through foreign key constraints and atomic transactions

Limitations

- Password hashing uses plain text in the prototype; bcrypt should be used in production
- No multi-location support - all data is scoped to a single gym

- No external payment gateway integration; payments are recorded manually
- No automated email or push notification system for overdue payments

XIV. Future Enhancements

Proposed Improvements

Security Enhancements

- Implement bcrypt password hashing
- Add JWT token-based authentication
- Introduce session expiry and token refresh

Advanced Features

- Automated email/SMS reminders for overdue payments and upcoming sessions
- Multi-location support with branch-specific data isolation
- Mobile app (iOS/Android) with real-time check-in via QR code
- Integration with external payment gateways (Stripe, JazzCash, EasyPaisa)

Analytics Enhancements

- Member retention and churn rate analysis
- Custom report builder for admins
- Predictive analytics for session demand forecasting
- Trainer performance scoring based on member progress

Database Improvements

- Database views (vw_member_dashboard, vw_session_capacity) for simplified querying
- Audit logging triggers to track data changes
- Read replicas for analytics workloads
- Automated backup scheduling

This system provides a solid, extensible foundation that can be scaled to meet the evolving needs of a real gym business while maintaining data integrity, security, and performance.